NE 723

Neutron Transport Theory

North Carolina State University
Department of Nuclear Engineering

Computer Project 2

Author

Joseph F. Specht IV

jfspecht

Professor

Dmitriy Y. Anistratov

November 10, 2026

Contents

1 Quasi-Diffusion (QD) Method **3**

1.1 Transport Problem . . . 3

1.2 Quasi-Diffusion Equations . . . 4

2 Algorithm Flowcharts **6**

2.1 Master Program Execution . . . 6

2.2 Quasi-Diffusion Method . . . 7

2.3 Angular Mesh Sweep . . . 8

2.4 Low-Order QD Solver . . . 9

3 Discretization Methods **10**

3.1 Lumped Linear Discontinuous (LLD) Scheme . . . 10

3.1.1 Standard Linear Discontinuous Scheme . . . 10

3.1.2 Lumped Linear Discontinuous Scheme . . . 11

3.1.3 Forward and Backward Sweep Equations . . . 12

3.2 Finite Volume (FV) Scheme . . . 14

3.2.1 Finite Volume Equations . . . 14

3.2.2 Linear System Formulation . . . 15

3.3 Projection Operator Approach . . . 18

3.3.1 Required and Returned Values by QD Solvers . . . 18

3.3.2 Projection Operator Approach . . . 19

3.3.3 Discrete Prolongation Operator . . . 19

4 Results **22**

4.1 Test A . . . 22

4.2 Test B . . . 24

4.3 Test C . . . 26

4.4 Test D(a) . . . 28

4.5 Test D(b) . . . 30

A Python Source Code **32**

A.1 Master Program (`main.py`) . . . 32

A.2 Spatial Mesh Discretization (`mesh_builder.py`) . . . 35

A.3 Angular Quadrature Discretization (`quadrature.py`) 37
A.4 Quasi-Diffusion and Transport Solvers (`solvers.py`) 38
A.5 Output and Logging Manager (`output_manager.py`) 46
A.6 Report Plotting Module (`plots.py`) 50

B Input Files **54**

B.1 Test A (`input_files/test_a.py`) 54
B.2 Test B (`input_files/test_b.py`) 54
B.3 Test C (`input_files/test_c.py`) 55
B.4 Test D(a) (`input_files/test_d1.py`) 55
B.5 Test D(b) (`input_files/test_d2.py`) 56

1 Quasi-Diffusion (QD) Method

In this section, I detail the Quasi-Diffusion (QD) method. A description of the QD method is not required, however, I took notes to ensure I understood the Quasi-Diffusion Method, so I will include them here.

1.1 Transport Problem

We consider the one-group, neutron, transport equation for 1D slab geometry.

$$\begin{gathered}\mathcal{L}\psi = \mathcal{S}\psi + Q, \\ 0 \leq x \leq X.\end{gathered} \tag{1}$$

With the transport term, $\mathcal{L}\psi$, defined as:

$$\mathcal{L}\psi(x,\mu) \equiv \mu\frac{\partial\psi(x,\mu)}{\partial x} + \Sigma_t(x)\psi(x,\mu), \tag{2}$$

the scattering term, $\mathcal{S}\psi$, defined as:

$$\mathcal{S}\psi(x,\mu) \equiv \frac{1}{2}\Sigma_s(x)\int_{-1}^{1}\psi(x,\mu')\,d\mu', \tag{3}$$

and the external source term, Q, given as:

$$Q = \frac{1}{2}q. \tag{4}$$

The boundary conditions for the problem are given as:

$$\psi(0,\mu) = \psi_{in}^{+}(\mu) \text{ for } \mu > 0 \tag{5}$$

$$\psi(X,\mu) = \psi_{in}^{-}(\mu) \text{ for } \mu < 0 \tag{6}$$

With the scalar flux and current defined as:

$$\phi(x) = \int_{-1}^{1}\psi(x,\mu)\,d\mu, \quad J(x) = \int_{-1}^{1}\mu\psi(x,\mu)\,d\mu, \tag{7}$$

we can write the high-order transport equation used in the QD method:

$$\mathcal{L}\psi = \frac{1}{2}\left(\Sigma_s\phi + q\right). \tag{8}$$

1.2 Quasi-Diffusion Equations

We can obtain more equations by taking moments of the high-order equation with respect to μ. When integrating over μ while taking the moment, the "high-order" equation loses its angular dependence and becomes a "low-order" equation. The n-th moment with respect to μ of some function f is given as follows:

$$\langle f \rangle_n = \int_{-1}^{1} \mu^n f(x,\mu)\, d\mu \tag{9}$$

We obtain the first low-order QD (LOQD) equation by taking the zeroth moment with respect to μ of the high-order transport equation. This first equation describes local neutron population balance – the net particle leakage plus absorption is equal to the external source.

$$\frac{d}{dx} J(x) + (\Sigma_t - \Sigma_s)\phi(x) = q(x) \tag{10}$$

We obtain the second LOQD by taking the first moment with respect to μ of the high-order transport equation. This second equation describes the local neutron current balance.

$$\frac{d}{dx}\left(E(x)\phi(x)\right) + \Sigma_t J(x) = 0 \tag{11}$$

The QD (or Eddington) factor, $E(x)$, is a dimensionless quantity calculated as below. The QD factor measures how directionally biased the neutron flight paths are. When there is an isotropic distribution of flight directions, the QD factor reduces to the 1/3 factor from diffusion theory.

$$E(x) = \frac{\int_{-1}^{1} \mu^2 \psi(x,\mu) d\mu}{\int_{-1}^{1} \psi(x,\mu) d\mu} \tag{12}$$

Additionally, we can rewrite the first moment equation and use knowledge from fluids to reinterpret the terms. The negative gradient of radiation pressure equals the spatial rate of current attenuation.

$$-\frac{d}{dx}\left(E(x)\phi(x)\right) = \Sigma_t J(x) \tag{13}$$

There are also Robin boundary conditions describing the current balance. Note, for a reflective boundary condition, $\psi(\mu) = \psi(-\mu)$, so the current equals zero.

$$J(0) = C_L\left(\phi(0) - \phi_{in}^{+}\right) + J_{in}^{+} \tag{14}$$

$$J(X) = C_R\left(\phi(X) - \phi_{in}^{-}\right) + J_{in}^{-} \tag{15}$$

The boundary factors, C_X, calculate the average, exiting μ at each boundary.

$$C_L = \frac{\int_{-1}^{0} \mu\psi(0,\mu)d\mu}{\int_{-1}^{0} \psi(0,\mu)d\mu} \tag{16}$$

$$C_R = \frac{\int_{0}^{1} \mu\psi(X,\mu)d\mu}{\int_{0}^{1} \psi(X,\mu)d\mu} \tag{17}$$

Therefore, we see the following:

$$J(0) = J_{out}^{-} + J_{in}^{+} = C_L\left(\phi(0) - \phi_{in}^{+}\right) + J_{in}^{+} \tag{18}$$

$$J_{out}^{-} = C_L\left(\phi(0) - \phi_{in}^{+}\right) \tag{19}$$

and

$$J(X) = J_{out}^{+} + J_{in}^{-} = C_R\left(\phi(X) - \phi_{in}^{-}\right) + J_{in}^{-} \tag{20}$$

$$J_{out}^{+} = C_R\left(\phi(X) - \phi_{in}^{-}\right) \tag{21}$$

Finally, define partial fluxes and currents:

$$\phi_{in}^{\pm} = \pm\int_{0}^{\pm 1} \psi_{in}^{\pm}(\mu)d\mu \tag{22}$$

$$J_{in}^{\pm} = \int_{0}^{\pm 1} \mu\psi_{in}^{\pm}(\mu)d\mu \tag{23}$$

2 Algorithm Flowcharts

2.1 Master Program Execution

Algorithm 1: Master Program Execution

File: `main.py`

Input:
1. Input file (`test_{a,b,c,d}.py`)
2. Execution mode flag `-mode` (`calc` , `plot`)

Output:
1. If `mode == "calc"` : output files containing plotted quantities, boundary factors, and iteration counts
2. If `mode == "plot"` : the ten required figures[1]

```
Import desired input file as problem and determine execution mode;
Set results directory path results_dir ;
if mode == "plot" then
    Load saved solution arrays from results_dir ;
    Generate and save figures;
    return
end
if mode == "calc" then
    Create results_dir ;
    Log problem setup;
    Discretize spatial mesh;
    Generate Gauss-Legendre quadrature set;
    Use the QD method to solve the fixed-source transport problem;
    Export desired output quantities;
    return
end
return
```

1. List of the ten required figures:

   - Cell-average angular flux $\psi_{i,m}$ versus (x_i, μ_m) (3D plot)
   - Cell-average scalar flux ϕ_i from the low-order QD problem versus x_i
   - Cell-edge scalar flux $\phi_{i+1/2}$ from the low-order QD problem versus $x_{i+1/2}$
   - Cell-edge current $J_{i+1/2}$ from the low-order QD problem versus $x_{i+1/2}$
   - Cell-average QD factor E_i versus x_i
   - Cell-edge QD factor $E_{i+1/2}$ versus $x_{i+1/2}$
   - Cell-average "departure from diffusion" transport metric $M_i^E = \left|\frac{1}{3} - E_i\right|$ versus x_i
   - Cell-average "QD factor gradient" transport gradient metric $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$ versus x_i
   - Convergence history $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$ versus iteration index s (log scale)
   - Estimated spectral radius $\rho^{(s)}$ versus iteration index s

2.2 Quasi-Diffusion Method

Algorithm 2: Quasi-Diffusion (QD) Method for the Fixed-Source Transport Problem

File: `solvers.qd_solver()`

Input:
1. Spatial mesh and material arrays (`mesh_dict`)
2. Gauss-Legendre quadrature set (`quad_dict`)
3. Boundary conditions
4. Convergence tolerance ϵ

Output:
1. Converged low-order grid-functions $(\phi_i, \phi_{i+1/2}, J_{i+1/2})$
2. Converged high-order angular flux $(\psi_{i,m})$
3. Converged QD factors
4. Converged boundary factors
5. Iteration history: (1) $L_\infty = \|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, (2) estimated spectral radius $\rho^{(s)}$, and (3) iterations required to converge

Initialize iteration index (`s=0`);
Start with guesses of: $\vec{\phi}^{(0)} = \vec{1}$, $E^{(1/2)} = 1/3$, $C_L^{(1/2)} = -1/2$, $C_R^{(1/2)} = 1/2$;
Initialize guesses for spectral radius $\rho_\phi^{(1)} = \rho_\phi^{(2)} = 1/2$;
The problem starts unconverged (`converged=False`);
while *not converged* **do**
    `s = s + 1` ;
    if *s > 1* **then**
        Perform a high-order LLD angular mesh sweep using $\vec{\phi}_i^{(s-1)}$;
        Compute the cell-average $E_i^{(s-1/2)}$ and the cell-edge $E_{i+1/2}^{(s-1/2)}$ QD factors;
        Compute the boundary factors $C_L^{(s-1/2)}$, $C_R^{(s-1/2)}$;
    end
    Solve the LOQD problem for $\phi_i^{(s)}, \phi_{i+1/2}^{(s)}$, and $J_{i+1/2}^{(s)}$ using $E^{(s-1/2)}$, $C_L^{(s-1/2)}$, and $C_R^{(s-1/2)}$;
    Apply discrete prolongation operator to the low-order scalar fluxes to obtain the scalar fluxes in the LLD space $\vec{\phi}_i^{(s)} = \mathcal{R}_h \left[\phi_{i-1/2}^{(s)} \quad \phi_i^{(s)} \quad \phi_{i+1/2}^{(s)}\right]^T$;
    if *s > 2* **then**
        Calculate the estimation of spectral radius $\rho_\phi^{(s)} = \frac{\|\phi^{(s)} - \phi^{(s-1)}\|_\infty}{\|\phi^{(s-1)} - \phi^{(s-2)}\|_\infty}$;
    end
    if $||\phi^{(s)} - \phi^{(s-1)}||_\infty \leq \varepsilon_\phi \left(\frac{1}{\rho_\phi^{(s)}} - 1\right)$ **then**
        `converged = True` ;
    end
end
Perform final angular mesh sweep using converged $\vec{\phi}_i^{(s)}$;
return *converged quantities and iteration history*

2.3 Angular Mesh Sweep

Algorithm 3: High-Order, Lumped Linear Discontinuous (LLD) Angular Mesh Sweep

File: `solvers.angular_mesh_sweep()`

Input:

1. Spatial mesh and material arrays (`mesh_dict`)
2. Gauss-Legendre quadrature set (`quad_dict`)
3. Boundary conditions
4. Previous, high-order, LLD scalar flux iterate $\vec{\phi}_i^{(s-1)} = (\phi_{L,i}^{(s-1)}, \phi_{R,i}^{(s-1)})^T$

Output:

1. Cell-average angular flux $\psi_{m,i}$ for all cells and angles
2. Cell-edge angular flux $\psi_{m,i+1/2}$ for all cells and angles

for $m = 1, \ldots, M$ **do**
  if $\mu_m > 0$ **then**
    Apply left boundary condition at $x_{1/2} = 0$ to set the incoming flux $\psi_{m,1/2}$;
    for $i = 1, \ldots, N$ **do**
      Construct the forward streaming and collision operator $\mathbf{A}_{m,i}$ and source vector $\vec{f}_{m,i}$[1];
      Solve the matrix equation $\mathbf{A}_{m,i}\vec{\psi}_{m,i} = \vec{f}_{m,i}$ for the cell-edge angular flux $\vec{\psi}_{m,i}$;
      Compute the cell-average angular flux $\psi_{m,i} = \frac{1}{2}(\psi_{L,m,i} + \psi_{R,m,i})$;
      Set $\psi_{m,i+1/2} = \psi_{R,m,i}$ for upwinding;
    end
  end
  if $\mu_m < 0$ **then**
    Apply right boundary condition at $x_{N+1/2} = L$ to set the incoming flux $\psi_{m,N+1/2}$;
    for $i = N, N-1, \ldots, 1$ **do**
      Construct the backward streaming and collision operator $\mathbf{A}_{m,i}$ and source vector $\vec{f}_{m,i}$[1] ;
      Solve the matrix equation $\mathbf{A}_{m,i}\vec{\psi}_{m,i} = \vec{f}_{m,i}$ for the cell-edge angular flux $\vec{\psi}_{m,i}$;
      Compute the cell-average angular flux $\psi_{m,i} = \frac{1}{2}(\psi_{L,m,i} + \psi_{R,m,i})$;
      Set $\psi_{m,i-1/2} = \psi_{L,m,i}$ for upwinding;
    end
  end
end
return *cell-average and cell-edge angular fluxes* $\psi_{m,i}$, $\psi_{m,i+1/2}$

1. The formulations of $\mathbf{A}_{m,i}$ and $\vec{f}_{m,i}$ depend on the current sweep direction. The derivation and formulations of $\mathbf{A}_{m,i}$ and $\vec{f}_{m,i}$ are presented in § 3.1.3.

2.4 Low-Order QD Solver

Algorithm 4: Low-Order Finite Volume Quasidiffusion (FV LOQD) Solver

File: `solvers.fv_loqd_solver()`

Input:

1. Spatial mesh and material arrays (`mesh_dict`)
2. Cell-average and cell-edge QD factors ($E_i^{(s-1/2)}, E_{i+1/2}^{(s-1/2)}$)
3. Boundary factors ($C_L^{(s-1/2)}, C_R^{(s-1/2)}$)
4. Boundary conditions

Output:

1. Low-order cell-average scalar flux $\phi_i^{(s)}$ for all cells
2. Low-order cell-edge scalar flux $\phi_{i+1/2}^{(s)}$ and current $J_{i+1/2}^{(s)}$

Assemble the interior rows of $\mathbf{A}$ using $E_i^{(s-1/2)}$, $E_{i+1/2}^{(s-1/2)}$, $\Sigma_{t,i}$, $\Sigma_{a,i}$, Δx_i, and q_i[1];
Assemble the interior entries of the source vector $\vec{f}$ using $E_i^{(s-1/2)}$, $E_{i+1/2}^{(s-1/2)}$, $\Sigma_{t,i}$, $\Sigma_{a,i}$, Δx_i, and q_i;
Use the left boundary condition to construct the first row of $\mathbf{A}$ and $\vec{f}$;
Use the right boundary condition to construct the last row of $\mathbf{A}$ and $\vec{f}$;
Solve $\mathbf{A}\vec{\phi}^{(s)} = \vec{f}$ for the cell-average scalar flux in each cell $\phi_i^{(s)}$;
Substitute $\phi_i^{(s)}$ to compute cell-edge scalar flux in each cell $\phi_{i+1/2}^{(s)}$;
Substitute $\phi_i^{(s)}$ and $\phi_{i+1/2}^{(s)}$ to compute cell-edge currents in each cell $J_{i+1/2}^{(s)}$;
return $\vec{\phi}_i^{(s)}$, $\vec{\phi}_{i+1/2}^{(s)}$, *and* $\vec{J}_{i+1/2}^{(s)}$

1. The discretized FV LOQD equations, tridiagonal matrix entries, boundary condition coefficients, and back-substitution expressions are derived in § 3.2.2.

3 Discretization Methods

3.1 Lumped Linear Discontinuous (LLD) Scheme

3.1.1 Standard Linear Discontinuous Scheme

In Computer Project 1, we used the linear discontinuous (LD) scheme for spatial discretization. I used the alternative form of the Linear DG method on Slide 18 of Lecture 10. We start with the definition of the angular-flux, finite-element expansion:

$$\psi_{m,i}(x) = \psi_{L,m,i} b_{L,i}(x) + \psi_{R,m,i} b_{R,i}(x), \tag{24}$$

where (i) $\psi_{m,i}$ is the cell-center (and cell-averaged as we use a linear method) angular flux for a discrete ordinate μ_m in cell i, (ii) $\psi_{\{L,R\},m,i}$ are the cell-edge angular fluxes for a discrete ordinate μ_m in cell i as approached from the cell center, and (iii) $b_{\{L,R\},i}$ are the Lagrange basis functions given by:

$$b_{L,i} = \frac{1}{\Delta x_i}(x_{i+1/2} - x), \quad b_{R,i} = \frac{1}{\Delta x_i}(x - x_{i-1/2}). \tag{25}$$

We are provided with the linear DG discretized transport equation for such a fixed source problem:

$$\mu_m \left(L_i^b \vec{\psi}_{m,i}^b + L_i \vec{\psi}_{m,i} \right) + \Sigma_{t,i} \Delta x_i M_i \vec{\psi}_{m,i} = \frac{1}{2} \Sigma_{s,i} \Delta x_i M_i \vec{\phi}_i + \frac{\Delta x_i}{2} M_i \vec{q}_i, \tag{26}$$

coefficient matrices:

$$L_i^b = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad L_i = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad M_i = \frac{1}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \tag{27}$$

angular flux:

$$\vec{\psi}_{m,i} = \left(\psi_{L,m,i}, \psi_{R,m,i} \right)^T, \tag{28}$$

scalar flux:

$$\vec{\phi}_i = \left(\phi_{L,i}, \phi_{R,i} \right)^T, \quad \phi_{c,i}^n = \sum_m \psi_{c,m,i}^n w_m, \tag{29}$$

external source:

$$\vec{q}_i = \left(q_{L,i}, q_{R,i} \right)^T. \tag{30}$$

and corresponding up-winding conditions:

$$\vec{\psi}^b_{m,i} = \left(\psi^b_{L,m,i}, \psi^b_{R,m,i}\right)^T, \tag{31}$$

$$\psi^b_{L,m,i} = \begin{cases} \psi_{L,m,i} & \text{if } \mu_m < 0, \\ \psi_{R,m,i-1} & \text{if } \mu_m > 0, \end{cases} \quad \psi^b_{R,m,i} = \begin{cases} \psi_{L,m,i+1} & \text{if } \mu_m < 0, \\ \psi_{R,m,i} & \text{if } \mu_m > 0. \end{cases} \tag{32}$$

3.1.2 Lumped Linear Discontinuous Scheme

In QD, we have a high-order equation, Eq. 8, and two low-order equations, Eqs. 10, 11. One of the advantages of QD is the high-order equation and low-order equations can have independent spatial discretization schemes. For this project, we use the lumped linear discontinuous (LLD) scheme to discretize the high-order equation, which is expounded further in this subsection. We will use a finite volume (FV) scheme to discretize the low-order equations, which is discussed in § 3.2.

The difference between the LD and the LLD schemes is how the spatial integrals used to calculate M_i are performed. Standard LD integrates basis function products analytically. For LD, the analytical integration yields an M_i with non-zero off-diagonal entries, meaning the cell edges are coupled.

Conversely, LLD uses a two-point trapezoidal quadrature at the cell edges instead of analytically integrating. The integral approximation "lumps" all the off-diagonal entries into the diagonal entry and sets the non-diagonal entries to zero. The lumping decouples the previously coupled cell edges as the off-diagonal entries of the mass matrix are now zero.

$$\int_{x_{i-1/2}}^{x_{i+1/2}} f(x) b_{c,i}(x)\, dx \approx \frac{1}{2} f\left(x_{i-1/2}\right) + \frac{1}{2} f\left(x_{i+1/2}\right) \tag{33}$$

$$M_i = \frac{1}{6}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \longrightarrow M_i^{\text{lumped}} = \frac{1}{2}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{34}$$

Using the lumped mass matrix to the previous LD equation yields the LLD scheme equations for our problem:

$$\mu_m \left(L_i^b \vec{\psi}_{m,i}^b + L_i \vec{\psi}_{m,i} \right) + \frac{1}{2} \Sigma_{t,i} \Delta x_i \vec{\psi}_{m,i} = \frac{\Delta x_i}{4} \left(\Sigma_{s,i} \vec{\phi}_i + \vec{q}_i \right). \tag{35}$$

The coefficient matrices, angular flux, scalar flux, external source, and up-winding equations do not change from LD to LLD. However, all LLD equations are provided below for completion:

$$L_i^b = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad L_i = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \tag{36}$$

angular flux:

$$\vec{\psi}_{m,i} = \left(\psi_{L,m,i}, \psi_{R,m,i} \right)^T, \tag{37}$$

scalar flux:

$$\vec{\phi}_i = \left(\phi_{L,i}, \phi_{R,i} \right)^T, \quad \phi_{c,i}^n = \sum_m \psi_{c,m,i}^n w_m, \tag{38}$$

external source:

$$\vec{q}_i = \left(q_{L,i}, q_{R,i} \right)^T. \tag{39}$$

and corresponding up-winding conditions:

$$\vec{\psi}_{m,i}^b = \left(\psi_{L,m,i}^b, \psi_{R,m,i}^b \right)^T, \tag{40}$$

$$\psi_{L,m,i}^b = \begin{cases} \psi_{L,m,i} & \text{if } \mu_m < 0, \\ \psi_{R,m,i-1} & \text{if } \mu_m > 0, \end{cases} \quad \psi_{R,m,i}^b = \begin{cases} \psi_{L,m,i+1} & \text{if } \mu_m < 0, \\ \psi_{R,m,i} & \text{if } \mu_m > 0. \end{cases} \tag{41}$$

3.1.3 Forward and Backward Sweep Equations

For our angular mesh sweep, we formulate the problem differently based on if we are sweeping forward or backward. We can manipulate the LLD scheme equations to obtain:

$$\left(\mu_m L_i + \frac{1}{2} \Sigma_{t,i} \Delta x_i \mathbf{I} \right) \vec{\psi}_{m,i} = \frac{\Delta x_i}{4} \left(\Sigma_{s,i} \vec{\phi}_i + \vec{q}_i \right) - \mu_m L_i^b \vec{\psi}_{m,i}^b \tag{42}$$

Note, if we have a reflective boundary condition, we sweeping from the side without the reflective boundary condition to the side with the reflective boundary condition.

Forward Sweep For the forward sweep or $\mu > 0$, the previous equation becomes:

$$\left(\mu_m L_i + \frac{1}{2}\Sigma_{t,i}\Delta x_i \mathbf{I}\right)\vec{\psi}_{m,i} = \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\vec{\phi}_i + \vec{q}_i\right) - \mu_m \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} (\psi_{R,m,i-1}, \psi_{R,m,i})^T \tag{43}$$

$$\left(\mu_m L_i + \frac{1}{2}\Sigma_{t,i}\Delta x_i \mathbf{I} + \mu_m \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}\right)\vec{\psi}_{m,i} = \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\vec{\phi}_i + \vec{q}_i\right) + \mu_m \begin{bmatrix} \psi_{R,m,i-1} \\ 0 \end{bmatrix} \tag{44}$$

When sweeping forward, we formulate the system as $\mathbf{A}_{m,i}\vec{\psi}_{m,i} = \vec{f}_{m,i}$ to obtain the following expressions:

$$\mathbf{A}_{m,i} = \frac{1}{2}\begin{bmatrix} \mu_m + \Sigma_{t,i}\Delta x_i & \mu_m \\ -\mu_m & \mu_m + \Sigma_{t,i}\Delta x_i \end{bmatrix} \tag{45}$$

$$\vec{f}_{m,i} = \begin{bmatrix} \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\phi_{L,i} + q_{L,i}\right) + \mu_m \psi_{R,m,i-1} \\ \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\phi_{R,i} + q_{R,i}\right) \end{bmatrix} \tag{46}$$

Backward Sweep For the backward sweep or $\mu < 0$, the manipulated equation becomes:

$$\left(\mu_m L_i + \frac{1}{2}\Sigma_{t,i}\Delta x_i \mathbf{I}\right)\vec{\psi}_{m,i} = \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\vec{\phi}_i + \vec{q}_i\right) - \mu_m \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} (\psi_{L,m,i}, \psi_{L,m,i+1})^T \tag{47}$$

$$\left(\mu_m L_i + \frac{1}{2}\Sigma_{t,i}\Delta x_i \mathbf{I} - \mu_m \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}\right)\vec{\psi}_{m,i} = \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\vec{\phi}_i + \vec{q}_i\right) - \mu_m \begin{bmatrix} 0 \\ \psi_{L,m,i+1} \end{bmatrix} \tag{48}$$

When sweeping backward, we formulate the system as $\mathbf{A}_{m,i}\vec{\psi}_{m,i} = \vec{f}_{m,i}$ to obtain the following expressions:

$$\mathbf{A}_{m,i} = \frac{1}{2}\begin{bmatrix} -\mu_m + \Sigma_{t,i}\Delta x_i & \mu_m \\ -\mu_m & -\mu_m + \Sigma_{t,i}\Delta x_i \end{bmatrix} \tag{49}$$

$$\vec{f}_{m,i} = \begin{bmatrix} \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\phi_{L,i} + q_{L,i}\right) \\ \frac{\Delta x_i}{4}\left(\Sigma_{s,i}\phi_{R,i} + q_{R,i}\right) - \mu_m \psi_{L,m,i+1} \end{bmatrix} \tag{50}$$

3.2 Finite Volume (FV) Scheme

The QD method allows for independent discretization of the high-order and low-order equations. I previously discussed the LLD scheme for the high-order equations. This subsection will present the finite volume (FV) discretization of the low-order equations.

3.2.1 Finite Volume Equations

Start by taking Eqs. 10,11 and discretizing these low-order equations using a second-order accurate FV scheme. We integrate the balance equation over the ith cell and we integrate the first moment equation twice over the left and right halves of the i^{th}cell. Following the integration, from Slide 12 of Lecture 17, the system of discretized, low-order QD equations are as follows:

balance equation:

$$J_{i+1/2} - J_{i-1/2} + (\Sigma_{t,i} - \Sigma_{s,i})\Delta x_i \phi_i = q_i \Delta x_i, \tag{51}$$

first moment equations:

$$E_i \phi_i - E_{i-1/2}\phi_{i-1/2} + \frac{1}{2}\Sigma_{t,i}\Delta x_i J_{i-1/2} = 0, \tag{52}$$

$$E_{i+1/2}\phi_{i+1/2} - E_i \phi_i + \frac{1}{2}\Sigma_{t,i}\Delta x_i J_{i+1/2} = 0, \tag{53}$$

and Robin boundary conditions:

$$J_{1/2} = C_L(\phi_{1/2} - \phi_{in}^+) + J_{in}^+, \tag{54}$$

$$J_{N+1/2} = C_R(\phi_{N+1/2} - \phi_{in}^-) + J_{in}^-. \tag{55}$$

Each our N cells has three associated equations in Eqs. 51,52,53. With the two boundary conditions, we have $3N + 2$ coupled equations. Note, if we have reflective boundary conditions at a boundary, the current equals zero at that boundary.

Applying the FV scheme to the remaining equations in § 1.2, we obtain more discretized equations required for our solution.

the incoming partial fluxes:

$$\phi_{in}^{+} = \sum_{\{m:\mu_m>0\}} \psi_{in,m}^{+} w_m, \quad \phi_{in}^{-} = \sum_{\{m:\mu_m<0\}} \psi_{in,m}^{-} w_m, \tag{56}$$

the incoming partial currents:

$$J_{in}^{+} = \sum_{\{m:\mu_m>0\}} \mu_m \psi_{in,m}^{+} w_m, \quad J_{in}^{-} = \sum_{\{m:\mu_m<0\}} \mu_m \psi_{in,m}^{-} w_m. \tag{57}$$

For our closure equations, we obtain the following cell-average and cell-edge QD factors:

$$E_i = \frac{\sum_{m=1}^{M} \mu_m^2 \psi_{m,i} w_m}{\sum_{m=1}^{M} \psi_{m,i} w_m}, \tag{58}$$

$$E_{i+1/2} = \frac{\sum_{m=1}^{M} \mu_m^2 \psi_{m,i+1/2} w_m}{\sum_{m=1}^{M} \psi_{m,i+1/2} w_m}, \tag{59}$$

and the boundary factors:

$$C_L = \frac{\sum_{\{m:\mu_m<0\}} \mu_m \psi_{m,1/2} w_m}{\sum_{\{m:\mu_m<0\}} \psi_{m,1/2} w_m}, \tag{60}$$

$$C_R = \frac{\sum_{\{m:\mu_m>0\}} \mu_m \psi_{m,N+1/2} w_m}{\sum_{\{m:\mu_m>0\}} \psi_{m,N+1/2} w_m}. \tag{61}$$

where $\psi_{m,i} = \frac{1}{2}(\psi_{L,m,i} + \psi_{R,m,i})$ is the cell-average angular flux from the LLD scheme, and $\psi_{m,i+1/2}$ is the upwinded cell-edge angular flux. Additionally, the upwinded flux is determined via the boundary conditions for the boundary cells.

3.2.2 Linear System Formulation

To numerically solve the FV scheme, we need to formulate the problem to be solved by a computer. First, rearrange the right-half-cell, first-moment equation (Eq. 53) to isolate the right-edge, cell-edge current:

$$J_{i+1/2} = \frac{2}{\Sigma_{t,i+1}\Delta x_{i+1}} \left(E_{i+1/2}\phi_{i+1/2} - E_{i+1}\phi_{i+1}\right) \tag{62}$$

Similarly, rearrange the left-half-cell, first-moment equation (Eq. 52) to isolate the left–cell-edge current

in the neighboring $i+1^{\text{th}}$ cell:

$$J_{i+1/2} = \frac{2}{\Sigma_{t,i}\Delta x_i}\left(E_i\phi_i - E_{i+1/2}\phi_{i+1/2}\right) \tag{63}$$

Equate the two previous equations to solve for the radiation pressure at the right-cell-edge:

$$E_{i+1/2}\phi_{i+1/2} = \frac{\Sigma_{t,i+1}\Delta x_{i+1}E_i\phi_i + \Sigma_{t,i}\Delta x_i E_{i+1}\phi_{i+1}}{\Sigma_{t,i}\Delta x_i + \Sigma_{t,i+1}\Delta x_{i+1}} \tag{64}$$

The result from the previous equation can be substituted back to obtain the right-cell-edge current using cell-average scalar fluxes.

$$J_{i+1/2} = \frac{2}{\Sigma_{t,i}\Delta x_i + \Sigma_{t,i+1}\Delta x_{i+1}}\left(E_i\phi_i - E_{i+1}\phi_{i+1}\right) \tag{65}$$

We use this formulation of the cell-edge current to write the balance equation (Eq. 51) for all cells as:

$$\mathbf{A}\vec{\phi} = \vec{f} \tag{66}$$

where $\mathbf{A}$ is a tridiagonal matrix. We use the boundary conditions to generate the first and last rows of $\mathbf{A}$. For the interior cells, the sub-diagonal ($a_{i,i-1}$), diagonal ($a_{i,i}$), and super-diagonal ($a_{i,i+1}$) entries are given below. Note, the interior cells are the cells with indices $1 < i < N$:

$$a_{i,i-1} = -\frac{2E_{i-1}}{\Sigma_{t,i-1}\Delta x_{i-1} + \Sigma_{t,i}\Delta x_i}\,, \tag{67}$$

$$a_{i,i} = \frac{2E_i}{\Sigma_{t,i-1}\Delta x_{i-1} + \Sigma_{t,i}\Delta x_i} + \left(\Sigma_{t,i} - \Sigma_{s,i}\right)\Delta x_i + \frac{2E_i}{\Sigma_{t,i+1}\Delta x_{i+1} + \Sigma_{t,i}\Delta x_i}\,, \tag{68}$$

$$a_{i,i+1} = -\frac{2E_{i+1}}{\Sigma_{t,i+1}\Delta x_{i+1} + \Sigma_{t,i}\Delta x_i}\,. \tag{69}$$

the cell-averaged scalar flux vector $\in \mathbb{R}^N$ where N is the number of cells:

$$\vec{\phi} = \begin{pmatrix}\phi_1 & \phi_2 & \dots & \phi_{N-1} & \phi_N\end{pmatrix}^T \tag{70}$$

and the i^{th}entry of the source vector is given as:

$$f_i = q_i\Delta x_i \tag{71}$$

For the first cell/row of $\mathbf{A}$, use the left boundary condition to determine the incoming flux. Substituting the expressions for the left and right cell-edge currents into the balance equation (Eq. 51) yields in the first row of $\mathbf{A}$ and $\vec{f}$ as:

$$a_{1,1} = \frac{2E_1}{\Sigma_{t,1}\Delta x_1 + \Sigma_{t,2}\Delta x_2} + \frac{C_L E_1}{E_{1/2} - \frac{1}{2}\Sigma_{t,1}\Delta x_1 C_L} + (\Sigma_{t,1} - \Sigma_{s,1})\,\Delta x_1\,, \tag{72}$$

$$a_{1,2} = -\frac{2E_2}{\Sigma_{t,1}\Delta x_1 + \Sigma_{t,2}\Delta x_2} \tag{73}$$

$$f_1 = q_1\Delta x_1 + \frac{E_{1/2}\left(J^+_{in} - C_L\phi^+_{in}\right)}{E_{1/2} - \frac{1}{2}\Sigma_{t,1}\Delta x_1 C_L} \tag{74}$$

Apply the same procedure using the right boundary condition to generate the entries of $\mathbf{A}$ and $\vec{f}$ for the Nth cell:

$$a_{N,N-1} = -\frac{2E_{N-1}}{\Sigma_{t,N-1}\Delta x_{N-1} + \Sigma_{t,N}\Delta x_N}\,, \tag{75}$$

$$a_{N,N} = \frac{2E_N}{\Sigma_{t,N-1}\Delta x_{N-1} + \Sigma_{t,N}\Delta x_N} + \frac{C_R E_N}{E_{N+1/2} + \frac{1}{2}\Sigma_{t,N}\Delta x_N C_R} + (\Sigma_{t,N} - \Sigma_{s,N})\,\Delta x_N\,, \tag{76}$$

$$f_N = q_N\Delta x_N - \frac{E_{N+1/2}\left(J^-_{in} - C_R\phi^-_{in}\right)}{E_{N+1/2} + \frac{1}{2}\Sigma_{t,N}\Delta x_N C_R}\,. \tag{77}$$

With the linear system fully constructed, use a linear solver to compute the cell-average scalar fluxes. I used `scipy.linalg.solve_banded()` to exploit the tri-diagonal structure of $\mathbf{A}$.

Cell-Edge Scalar Fluxes and Currents With the cell-average scalar fluxes $\vec{\phi}$, we can recover the cell-edge scalar fluxes $\phi_{i+1/2}$ and the cell-edge currents $J_{i+1/2}$ for the cell edges interior to the problem domain.

For the interior cell edge, we divide the known radiation pressure by the QD factor at the edge to obtain the cell-edge scalar flux:

$$\phi_{i+1/2} = \frac{\Sigma_{t,i+1}\Delta x_{i+1}E_i\phi_i + \Sigma_{t,i}\Delta x_i E_{i+1}\phi_{i+1}}{E_{i+1/2}\left(\Sigma_{t,i}\Delta x_i + \Sigma_{t,i+1}\Delta x_{i+1}\right)}\,, \tag{78}$$

We can directly evaluate the cell-edge current:

$$J_{i+1/2} = \frac{2}{\Sigma_{t,i}\Delta x_i + \Sigma_{t,i+1}\Delta x_{i+1}}\left(E_i\phi_i - E_{i+1}\phi_{i+1}\right)\,. \tag{79}$$

Substituting the appropriate boundary conditions into the half-cell, first-moment equations yields the

boundary scalar fluxes and currents:

$$\phi_{1/2} = \frac{E_1\phi_1 + \frac{1}{2}\Sigma_{t,1}\Delta x_1\left(J_{in}^{+} - C_L\phi_{in}^{+}\right)}{E_{1/2} - \frac{1}{2}\Sigma_{t,1}\Delta x_1 C_L}, \qquad J_{1/2} = \frac{C_L E_1\phi_1 + E_{1/2}\left(J_{in}^{+} - C_L\phi_{in}^{+}\right)}{E_{1/2} - \frac{1}{2}\Sigma_{t,1}\Delta x_1 C_L}, \tag{80}$$

$$\phi_{N+1/2} = \frac{E_N\phi_N - \frac{1}{2}\Sigma_{t,N}\Delta x_N\left(J_{in}^{-} - C_R\phi_{in}^{-}\right)}{E_{N+1/2} + \frac{1}{2}\Sigma_{t,N}\Delta x_N C_R}, \qquad J_{N+1/2} = \frac{C_R E_N\phi_N + E_{N+1/2}\left(J_{in}^{-} - C_R\phi_{in}^{-}\right)}{E_{N+1/2} + \frac{1}{2}\Sigma_{t,N}\Delta x_N C_R}. \tag{81}$$

3.3 Projection Operator Approach

This subsection discusses the projection operator approach used to solve the problem. The discrete prolongation operator is discussed in § 3.3.3.

3.3.1 Required and Returned Values by QD Solvers

The high-order and low-order equations are discretized on independent spatial meshes. As such, the spatial representation of the scalar flux is also independent requiring a mapping between the high-order and low-order scalar flux grid functions. To understand the mapping, it is important to understand which values are required for and calculated by each solver.

- **LLD HOQD Solver:**

  Requires:

  1. $\vec{\phi}_i = (\phi_{L,i},\, \phi_{R,i})^T$ – the *discontinuous* cell-edge scalar fluxes,

  Returns:

  1. $\vec{\psi}_{m,i} = (\psi_{L,m,i},\, \psi_{R,m,i})^T$ – the *discontinuous* cell-edge angular fluxes.

- **FV LOQD Solver:**

  Requires:

  1. E_i – the cell-average QD factor,
  2. $E_{i+1/2}$ – the cell-edge QD factor,
  3. C_L, C_R – the left and right boundary factors.

  Returns:

  1. ϕ_i – the cell-average scalar flux,
  2. $\phi_{i+1/2}$ – the *continuous* cell-edge scalar fluxes,
  3. $J_{i+1/2}$ – the *continuous* cell-edge currents.

3.3.2 Projection Operator Approach

For the projection operator approach, we need three operators. The first operator $\mathcal{P}$ is a projection operator that projects the high-order equation into the low-order moment space to generate the low-order equations. The second operator $\mathcal{F}$ is another projection operator that projects the high-order angular flux onto its angular moments, ϕ and J. The final operator $\mathcal{R}$ is a prolongation operator that reconstructs the high-order solution using the low-order solutions.

In § 1.2, we used $\mathcal{P}$ (Eq. 9) to generate the low-order equations and $\mathcal{F}$ to obtain the angular moments of the high-order solution (Eq. 7). The resultant low-order equations and angular moments are continuous, low-order quantities. After applying the projection operators, we discretize the high-order and low-order equations independently. The projection operators are continuous as the projection is applied to continuous functions before independent spatial discretization. However, the prolongation operator $\mathcal{R}$ requires further discussion as we need a discretized form of the prolongation operator.

3.3.3 Discrete Prolongation Operator

This subsection discusses the discrete prolongation operator of the QD method on the spatial mesh which defines mapping of the grid functions of the scalar flux from the discretized LOQD equations to the grid functions of the discrete scalar flux in the scattering term of the LLD scheme for the high-order transport equation.

The discrete prolongation operator $\mathcal{R}_h$ maps the low-order scalar flux to the high-order flux. $\mathcal{R}_h$ uses the FV cell-averaged scalar flux (ϕ_i) with the continuous, cell-edge, scalar fluxes ($\phi_{i-1/2}, \phi_{i+1/2}$) to determine the discontinuous, cell-edge, scalar fluxes for the LLD scheme. We also impose two constraints for the reconstructed flux calculated by $\mathcal{R}$.

Neutron Population Conservation First, the total number of particles in each cell should be conserved. We integrate the scalar flux over the i^{th}cell to obtain the total neutron population in the i^{th}cell:

$$\phi_i(x) = \phi_{L,i} b_{L,i}(x) + \phi_{R,i} b_{R,i}(x) \tag{82}$$

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \phi_i(x) dx = \frac{\Delta x_i}{2} \left(\phi_{L,i} + \phi_{R,i}\right) \tag{83}$$

We know the cell-average scalar flux ϕ_i from the FV LOQD solver times the width of the cell also equals

the total neutron population in the i^{th}cell. We equate the two total neutron population expression:

$$\Delta x_i \phi_i = \frac{\Delta x_i}{2} \left(\phi_{L,i} + \phi_{R,i} \right) \tag{84}$$

$$\phi_i = \frac{1}{2} \left(\phi_{L,i} + \phi_{R,i} \right) \tag{85}$$

Equating Spatial Gradients The neutron population conservation constraint is a zeroth-order, spatial-moment condition. The zeroth-order condition ensures a first-order-accurate, prolongation operator. To obtain a second-order-accurate prolongation operator, we need to impose a first-order, spatial-moment condition on the scalar flux.

We constrain $\mathcal{R}$ such that the discrete spatial gradients of the continuous (FV) scalar flux and discontinuous (LLD) scalar flux are equal in each cell. We represent the approximate gradient of the continuous scalar flux in the i^{th}cell using a full-cell, central, finite, difference method:

$$\frac{d\phi}{dx}\Big|_{x_i} \approx \frac{1}{\Delta x} \left(\phi_{i+1/2} - \phi_{i-1/2} \right) \tag{86}$$

We can take the spatial derivative of the LLD, cell-average, scalar flux (Eq. 82) to obtain the exact expression of the spatial gradient of the LLD, cell-average, scalar flux:

$$\frac{d\phi_i(x)}{dx} = \frac{1}{\Delta x_i} \left(\phi_{R,i} - \phi_{L,i} \right) \tag{87}$$

Equating the two expressions for the spatial gradients yields the following constraint:

$$\phi_{R,i} - \phi_{L,i} = \phi_{i+1/2} - \phi_{i-1/2} \tag{88}$$

Discrete Prolongation Operator Using the two scalar flux constraints, we define a second-order-accurate, discrete, prolongation operator $\mathcal{R}_h \in \mathbb{R}^{2\times 3}$, such that the $\mathcal{R}_h$ prolongs the low-order scalar flux to obtain the high-order scalar flux:

$$\mathcal{R}_h \begin{bmatrix} \phi_{i-1/2} \\ \phi_i \\ \phi_{i+1/2} \end{bmatrix} = \begin{bmatrix} \phi_{L,i} \\ \phi_{R,i} \end{bmatrix} = \vec{\phi}_i \tag{89}$$

$\mathcal{R}_h$ is unitless as $\mathcal{R}_h$ maps from scalar flux to scalar flux. Also, a non-trivial prolongation operator is required for independent spatial discretization of the high-order and low-order equations. With consistent discretization, $\mathcal{R}_h$ becomes an $q \times q$ identity matrix, where q is the number of unknowns per cell. We expect

$\mathcal{R}_h$ to be constant as the prolongation equality (Eq. 89) holds for every cell. In fact, the entries of $\mathcal{R}_h$ are determined purely from the two spatial discretization schemes.

To determine the value of $\mathcal{R}_h$, start by adding and subtracting the constraints on $\mathcal{R}_h$ together to obtain the following system of equations:

$$\begin{bmatrix} \phi_{L,i} \\ \\ \phi_{R,i} \end{bmatrix} = \begin{bmatrix} \phi_i - \dfrac{1}{2}\left(\phi_{i+1/2} - \phi_{i-1/2}\right) \\ \\ \phi_i + \dfrac{1}{2}\left(\phi_{i+1/2} - \phi_{i-1/2}\right) \end{bmatrix} = \mathcal{R}_h \begin{bmatrix} \phi_{i-1/2} \\ \\ \phi_i \\ \\ \phi_{i+1/2} \end{bmatrix} \tag{90}$$

We recognize the middle vector can be rewritten and factored:

$$\begin{bmatrix} \phi_i - \dfrac{1}{2}\left(\phi_{i+1/2} - \phi_{i-1/2}\right) \\ \\ \phi_i + \dfrac{1}{2}\left(\phi_{i+1/2} - \phi_{i-1/2}\right) \end{bmatrix} = \begin{bmatrix} \dfrac{1}{2}\phi_{i-1/2} + \phi_i - \dfrac{1}{2}\phi_{i+1/2} \\ \\ -\dfrac{1}{2}\phi_{i-1/2} + \phi_i + \dfrac{1}{2}\phi_{i+1/2} \end{bmatrix} = \begin{bmatrix} 1/2 & 1 & -1/2 \\ \\ -1/2 & 1 & 1/2 \end{bmatrix} \begin{bmatrix} \phi_{i-1/2} \\ \\ \phi_i \\ \\ \phi_{i+1/2} \end{bmatrix} \tag{91}$$

From the previous equation, we obtain the following discrete, prolongation operator for our combination of spatial discretization schemes:

$$\mathcal{R}_h = \begin{bmatrix} 1/2 & 1 & -1/2 \\ \\ -1/2 & 1 & 1/2 \end{bmatrix} \tag{92}$$

4 Results

4.1 Test A

Table 1: Converged iteration count and boundary factors for Test A.

Iterations (s)	C_L	C_R
9	−0.80024176	0.57283480

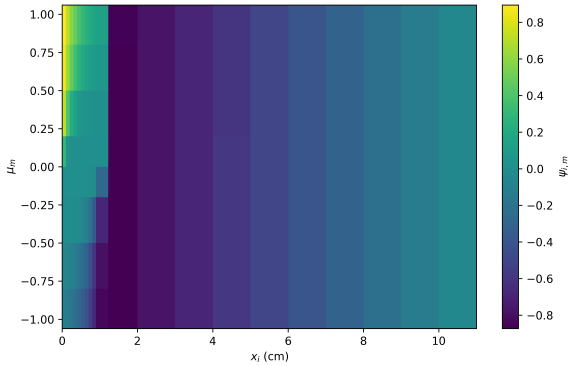


Figure 1: Cell-average values of angular flux, $\psi_{i,m}$, vs. (x_i, μ_m).

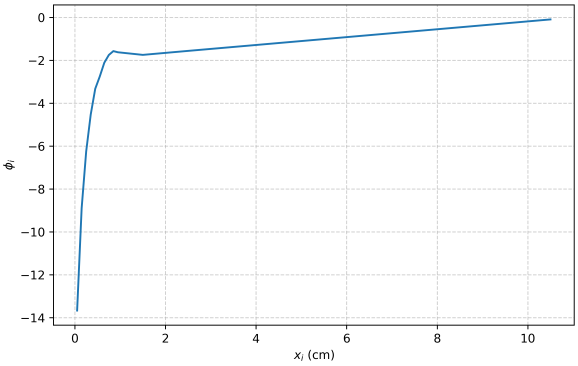


Figure 2: Cell-average values of scalar flux, ϕ_i, vs. x_i.

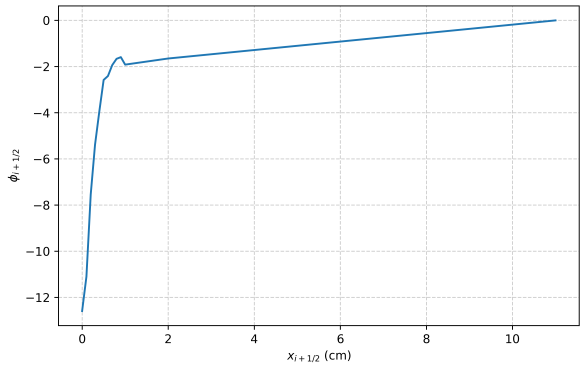


Figure 3: Cell-edge values of scalar flux, $\phi_{i+1/2}$, vs. $x_{i+1/2}$.

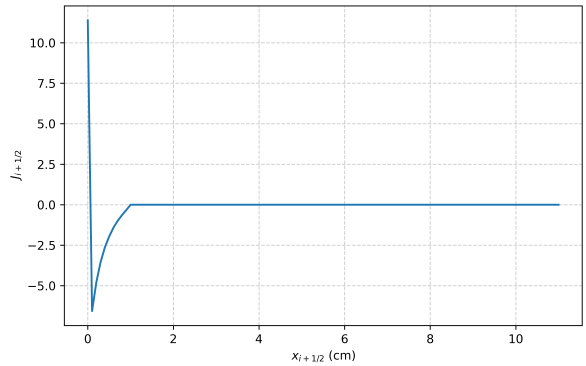


Figure 4: Cell-edge values of current, $J_{i+1/2}$, vs. $x_{i+1/2}$.

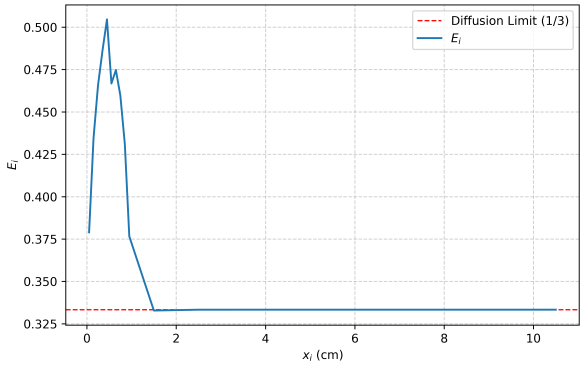


Figure 5: Cell-average values of QD factor, E_i, vs. x_i.

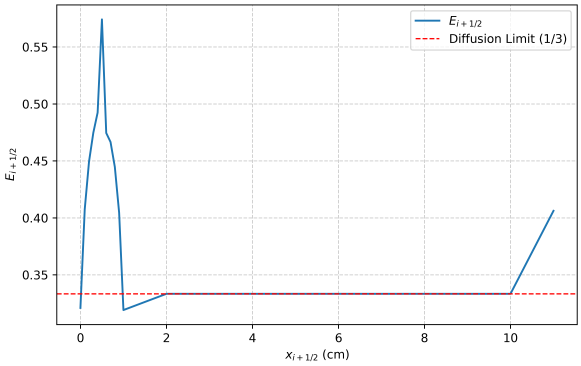


Figure 6: Cell-edge values of QD factor, $E_{i+1/2}$, vs. $x_{i+1/2}$.

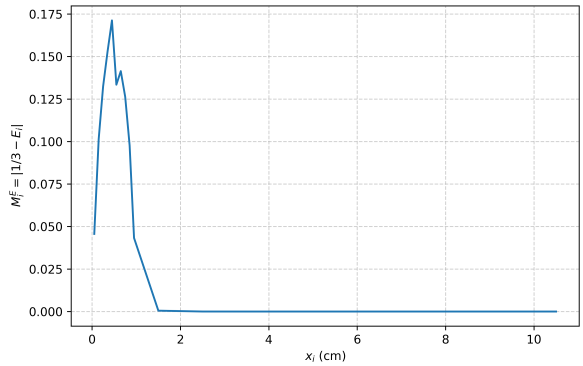


Figure 7: Cell-average values of transport metric, $M_i^E = \left|\frac{1}{3} - E_i\right|$, vs. x_i.

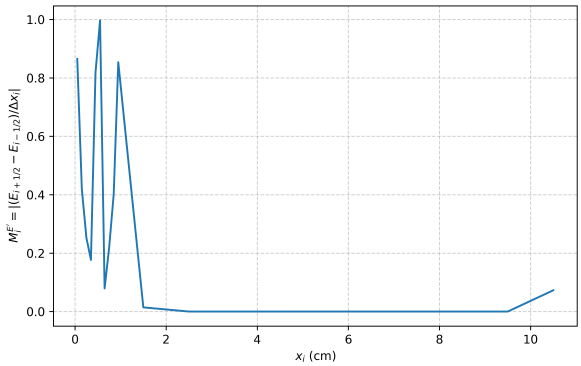


Figure 8: Cell-average values of transport gradient metric, $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$, vs. x_i.

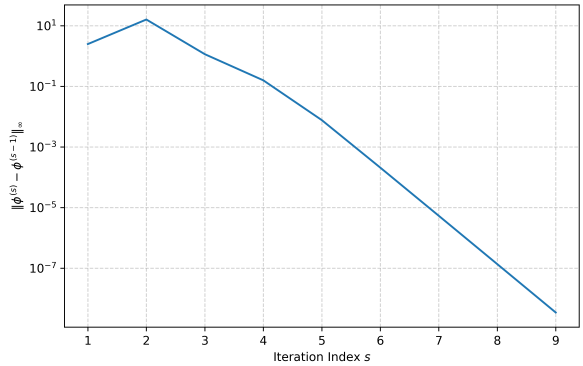


Figure 9: Convergence history, $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, vs. iteration index s.

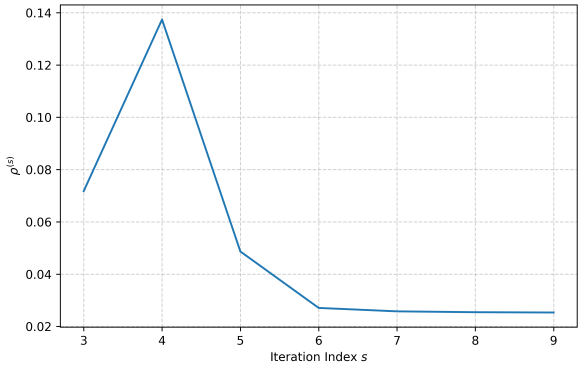


Figure 10: Estimated spectral radius, $\rho^{(s)}$, vs. iteration index s.

4.2 Test B

Table 2: Converged iteration count and boundary factors for Test B.

Iterations (s)	C_L	C_R
9	-0.50015539	0.53197945

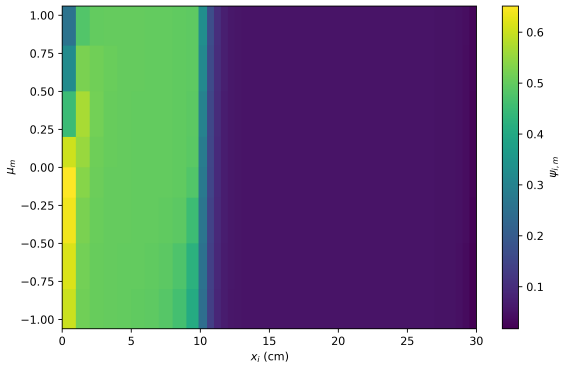


Figure 11: Cell-average values of angular flux, $\psi_{i,m}$, vs. (x_i, μ_m).

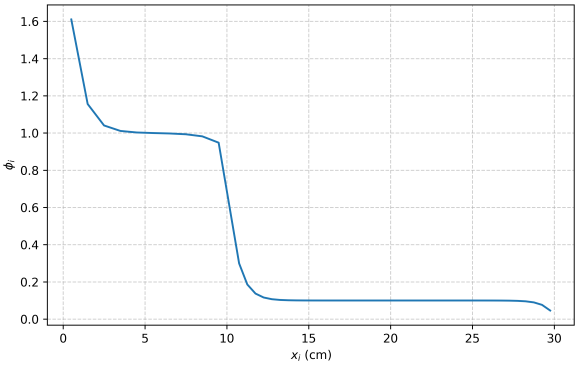


Figure 12: Cell-average values of scalar flux, ϕ_i, vs. x_i.

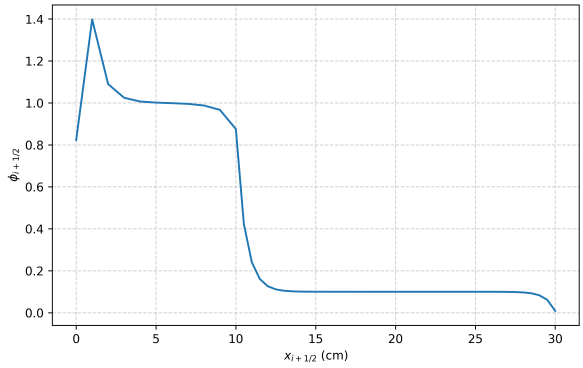


Figure 13: Cell-edge values of scalar flux, $\phi_{i+1/2}$, vs. $x_{i+1/2}$.

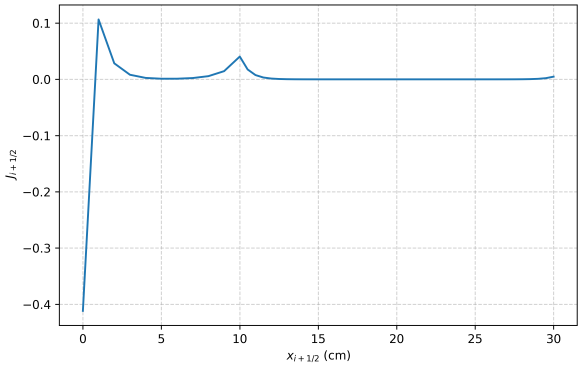


Figure 14: Cell-edge values of current, $J_{i+1/2}$, vs. $x_{i+1/2}$.

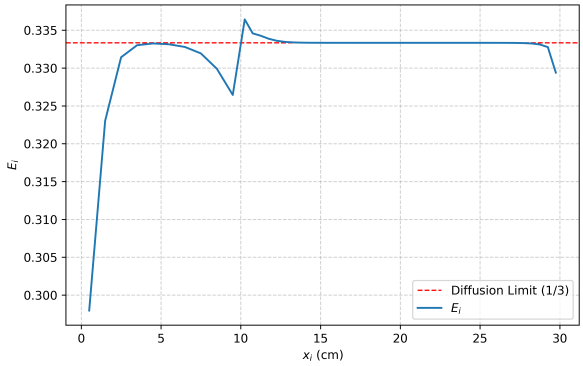


Figure 15: Cell-average values of QD factor, E_i, vs. x_i.

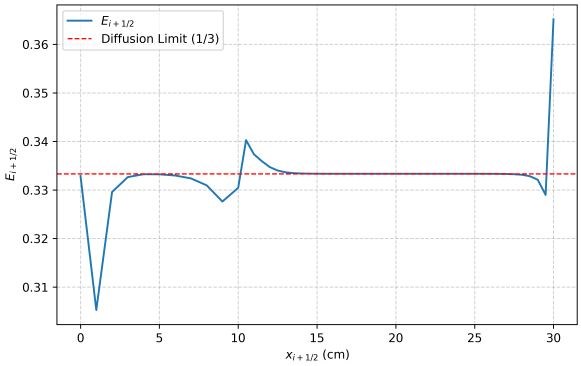


Figure 16: Cell-edge values of QD factor, $E_{i+1/2}$, vs. $x_{i+1/2}$.

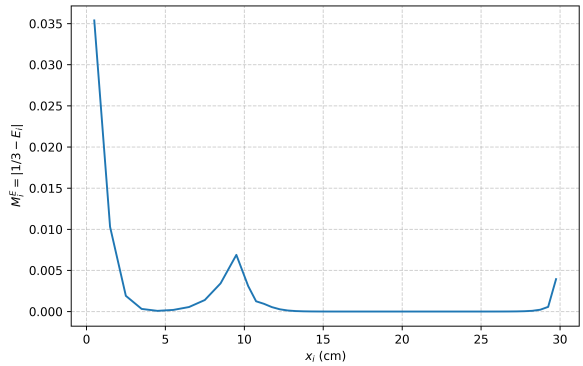


Figure 17: Cell-average values of transport metric, $M_i^E = \left|\frac{1}{3} - E_i\right|$, vs. x_i.

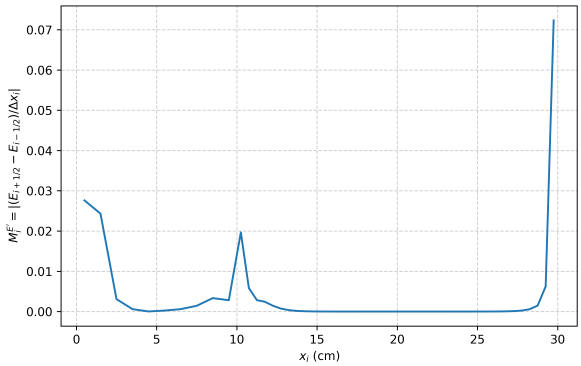


Figure 18: Cell-average values of transport gradient metric, $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$, vs. x_i.

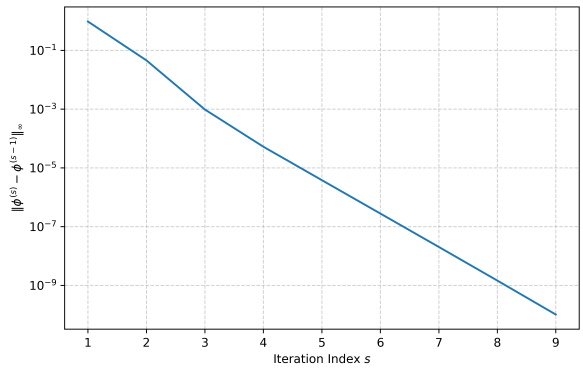


Figure 19: Convergence history, $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, vs. iteration index s.

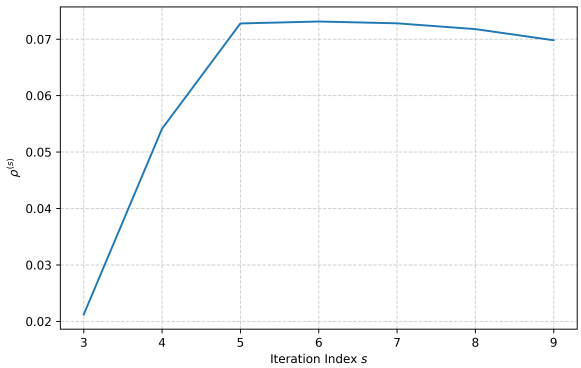


Figure 20: Estimated spectral radius, $\rho^{(s)}$, vs. iteration index s.

4.3 Test C

Table 3: Converged iteration count and boundary factors for Test C.

Iterations (s)	C_L	C_R
8	-0.50015513	0.64831183

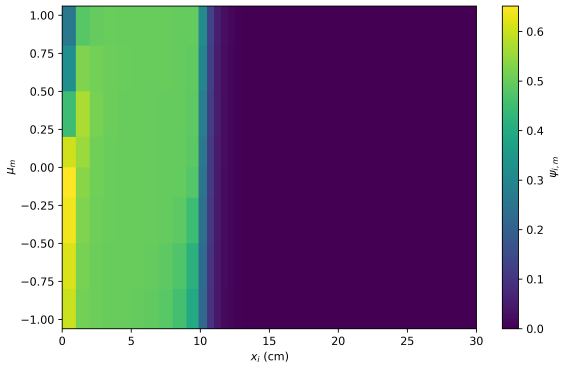


Figure 21: Cell-average values of angular flux, $\psi_{i,m}$, vs. (x_i, μ_m).

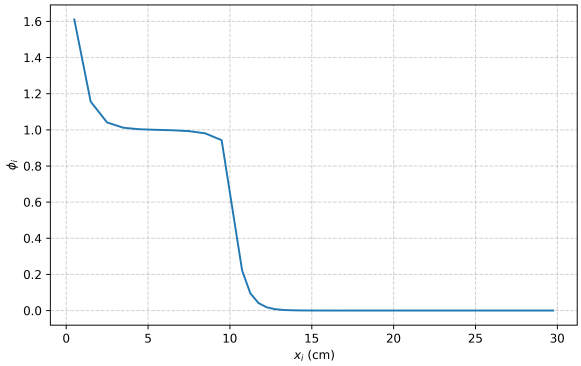


Figure 22: Cell-average values of scalar flux, ϕ_i, vs. x_i.

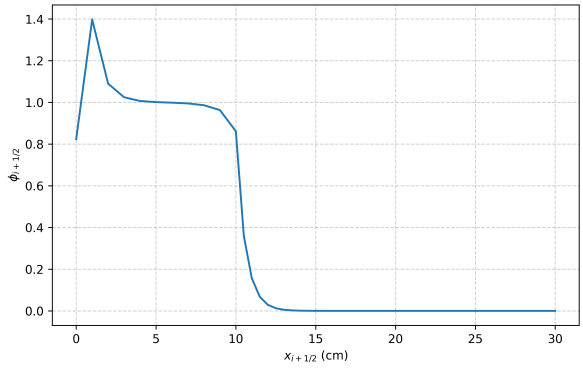


Figure 23: Cell-edge values of scalar flux, $\phi_{i+1/2}$, vs. $x_{i+1/2}$.

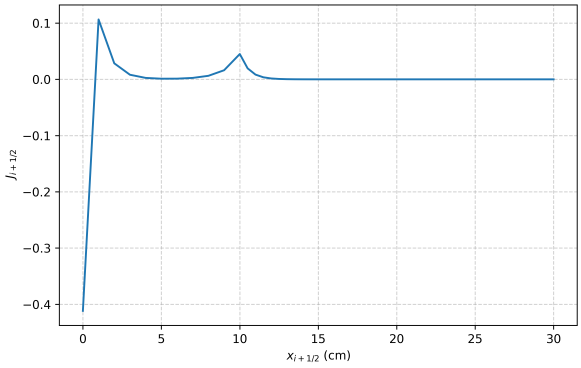


Figure 24: Cell-edge values of current, $J_{i+1/2}$, vs. $x_{i+1/2}$.

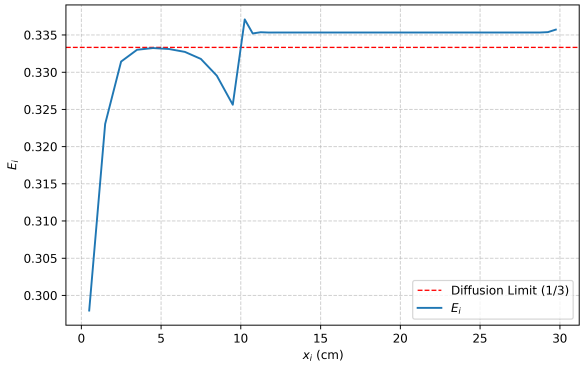


Figure 25: Cell-average values of QD factor, E_i, vs. x_i.

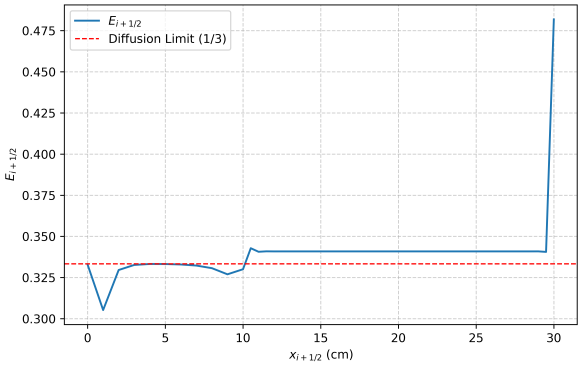


Figure 26: Cell-edge values of QD factor, $E_{i+1/2}$, vs. $x_{i+1/2}$.

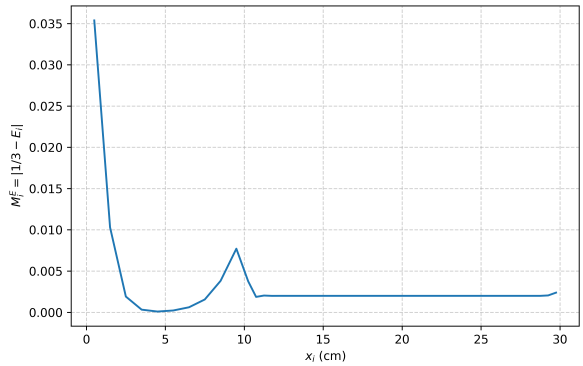


Figure 27: Cell-average values of transport metric, $M_i^E = \left|\frac{1}{3} - E_i\right|$, vs. x_i.

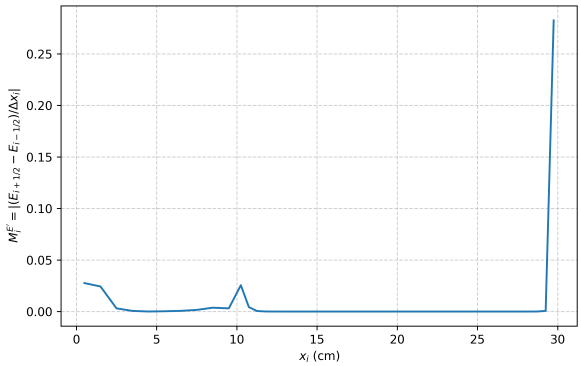


Figure 28: Cell-average values of transport gradient metric, $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$, vs. x_i.

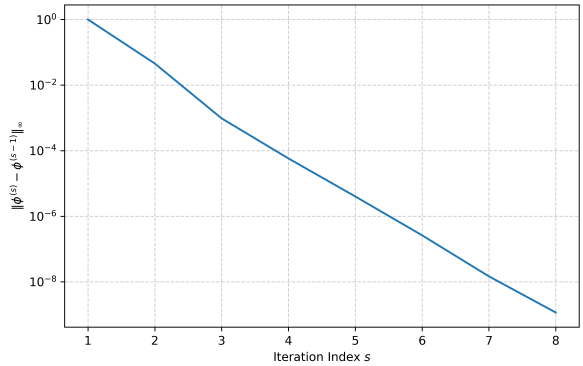


Figure 29: Convergence history, $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, vs. iteration index s.

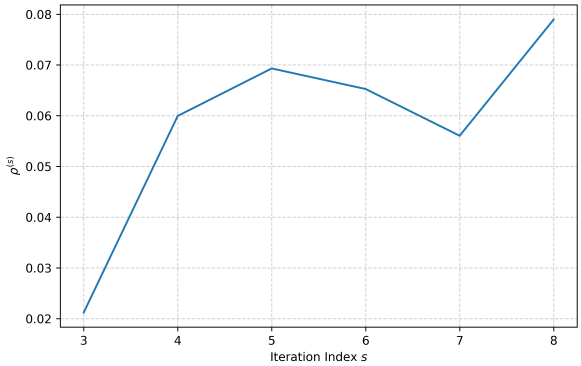


Figure 30: Estimated spectral radius, $\rho^{(s)}$, vs. iteration index s.

4.4 Test D(a)

Table 4: Converged iteration count and boundary factors for Test D(a) ($\nu = 10^{-2}$).

Iterations (s)	C_L	C_R
6	-0.50025567	0.54194759

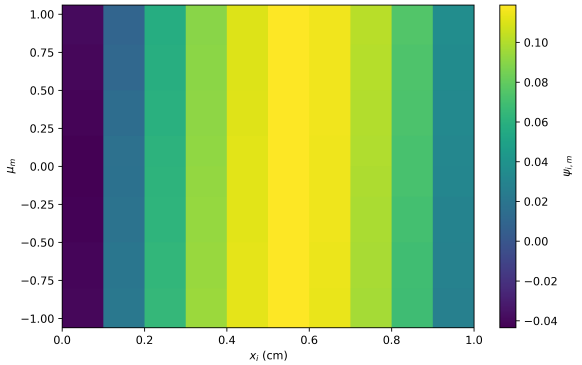


Figure 31: Cell-average values of angular flux, $\psi_{i,m}$, vs. (x_i, μ_m).

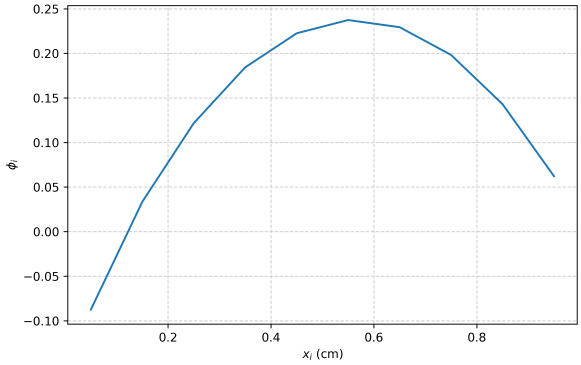


Figure 32: Cell-average values of scalar flux, ϕ_i, vs. x_i.

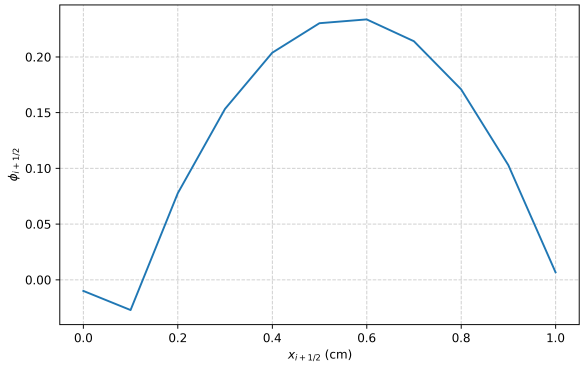


Figure 33: Cell-edge values of scalar flux, $\phi_{i+1/2}$, vs. $x_{i+1/2}$.

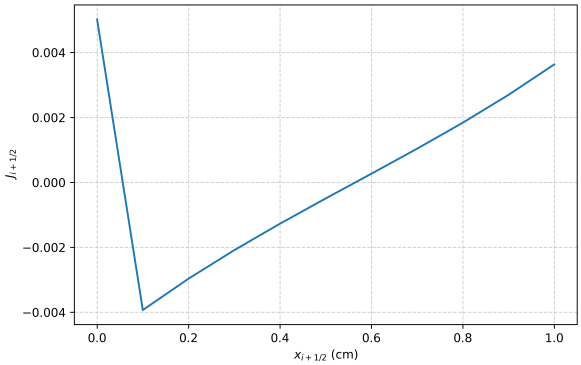


Figure 34: Cell-edge values of current, $J_{i+1/2}$, vs. $x_{i+1/2}$.

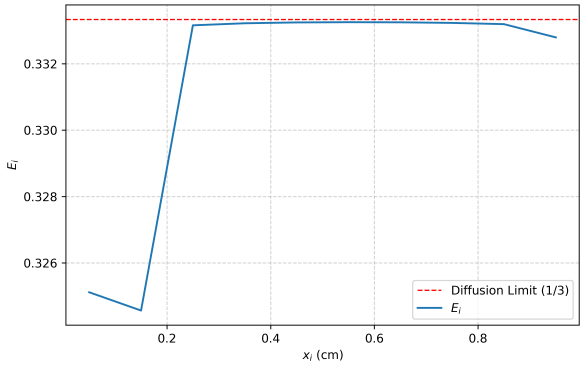


Figure 35: Cell-average values of QD factor, E_i, vs. x_i.

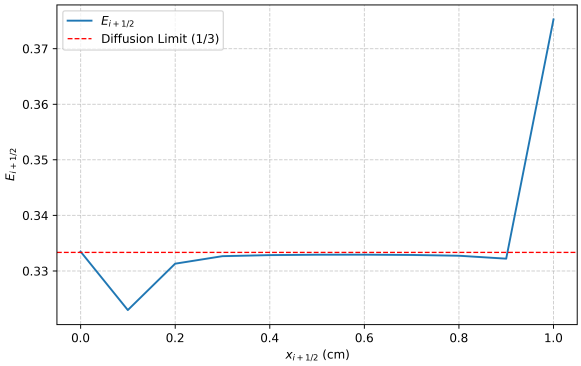


Figure 36: Cell-edge values of QD factor, $E_{i+1/2}$, vs. $x_{i+1/2}$.

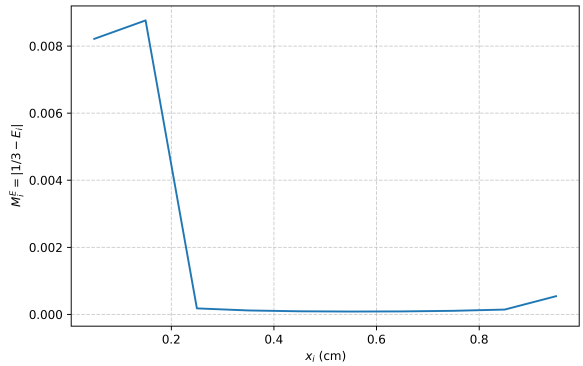


Figure 37: Cell-average values of transport metric, $M_i^E = \left|\frac{1}{3} - E_i\right|$, vs. x_i.

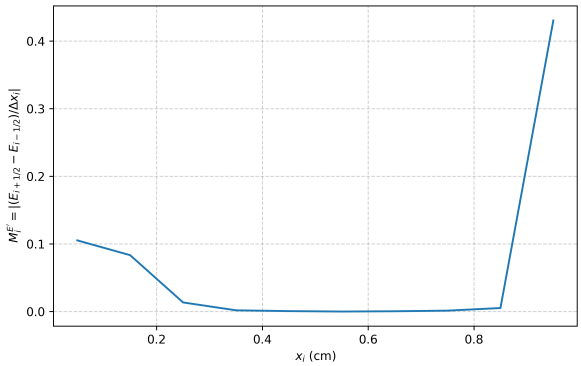


Figure 38: Cell-average values of transport gradient metric, $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$, vs. x_i.

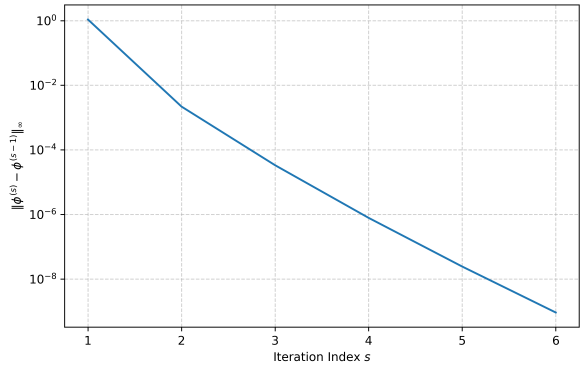


Figure 39: Convergence history, $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, vs. iteration index s.

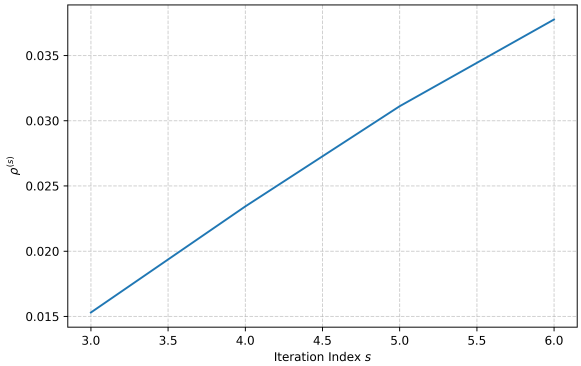


Figure 40: Estimated spectral radius, $\rho^{(s)}$, vs. iteration index s.

4.5 Test D(b)

Table 5: Converged iteration count and boundary factors for Test D(b) ($\nu = 10^{-4}$).

Iterations (s)	C_L	C_R
3	−0.50002018	0.50112278

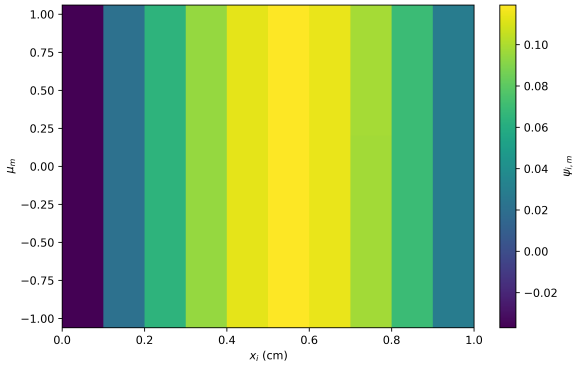


Figure 41: Cell-average values of angular flux, $\psi_{i,m}$, vs. (x_i, μ_m).

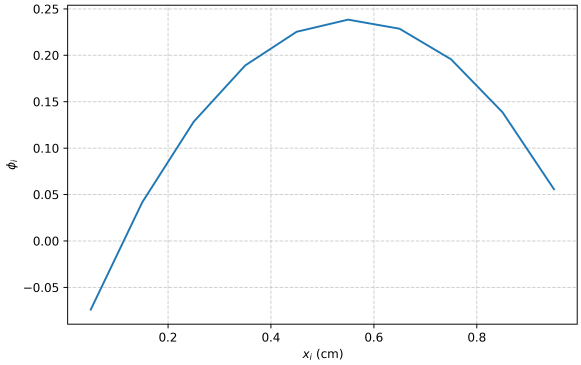


Figure 42: Cell-average values of scalar flux, ϕ_i, vs. x_i.

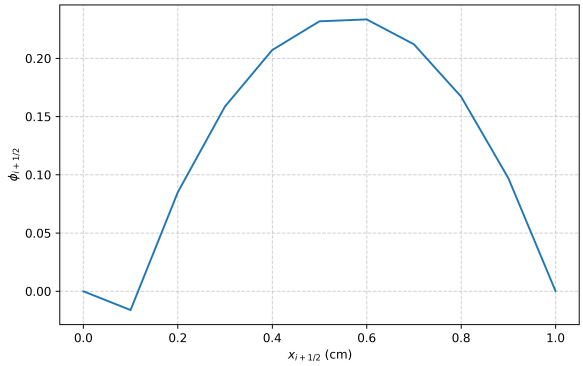


Figure 43: Cell-edge values of scalar flux, $\phi_{i+1/2}$, vs. $x_{i+1/2}$.

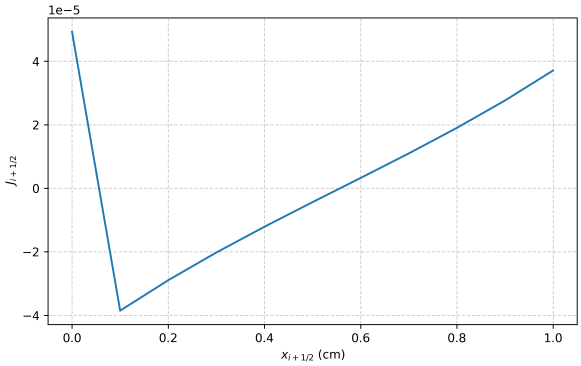


Figure 44: Cell-edge values of current, $J_{i+1/2}$, vs. $x_{i+1/2}$.

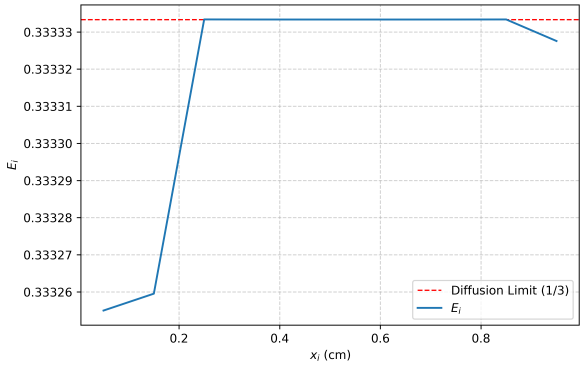


Figure 45: Cell-average values of QD factor, E_i, vs. x_i.

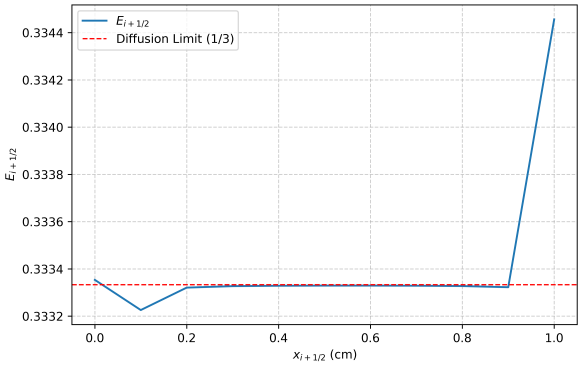


Figure 46: Cell-edge values of QD factor, $E_{i+1/2}$, vs. $x_{i+1/2}$.

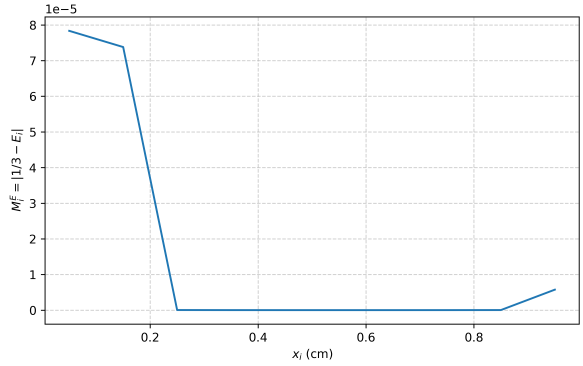


Figure 47: Cell-average values of transport metric, $M_i^E = \left|\frac{1}{3} - E_i\right|$, vs. x_i.

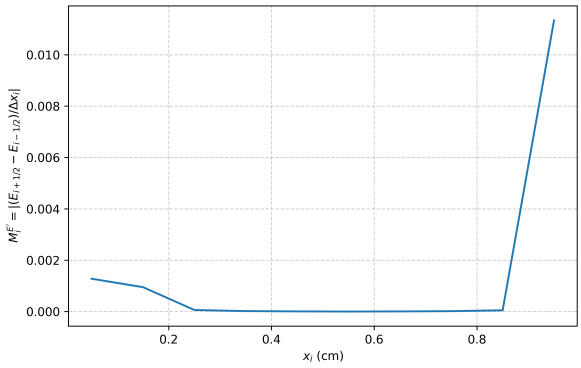


Figure 48: Cell-average values of transport gradient metric, $M_i^{E'} = \left|\frac{E_{i+1/2} - E_{i-1/2}}{\Delta x_i}\right|$, vs. x_i.

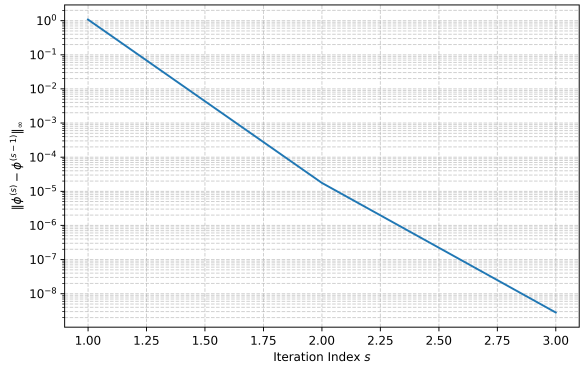


Figure 49: Convergence history, $\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$, vs. iteration index s.

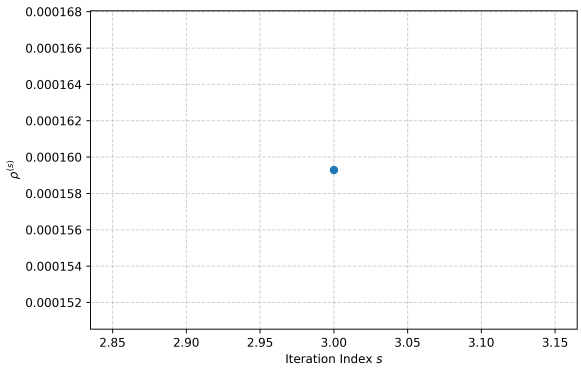


Figure 50: Estimated spectral radius, $\rho^{(s)}$, vs. iteration index s.

A Python Source Code

A.1 Master Program (main.py)

```python
# choose problem by uncommenting the desired import
from input_files import test_a as problem
#from input_files import test_b as problem
#from input_files import test_c as problem
#from input_files import test_d1 as problem
#from input_files import test_d2 as problem


# !!! you can switch the mode of this file to run the plotting feature !!!
#execution_mode = "calc"
execution_mode = "plot"

# CALCULATION MODE (calc)
#  - calc mode is the default mode
#  - running `python main.py` executes uncommented test (imported as problem)

# PLOTTING MODE (plot)
#  - you can run plot mode by passing a command line argument
#  - `python main.py --mode plot` will execute the script in the plotting mode


# package imports
import numpy as np
from pathlib import Path
import time
import sys

# module imports
import mesh_builder
import output_manager
import plots
import quadrature
import solvers


def main():
    start_time = time.time()
```

```
38
39      # setting up results path
40      base_results_dir = Path("results")
41      results_dir = base_results_dir / f"{problem.problem_name}"
42
43      # initial mode checks
44      if execution_mode not in ["calc", "plot"]:
45          msg = f"Unsupported execution mode ‘{exeuction_mode}‘. Use mode = [’calc’, ’plot’]"
46          raise ValueError(msg)
47
48      if execution_mode == "plot":
49          print(f"\n!!! Executing in ‘PLOT‘ !!!")
50
51          if not results_dir.exists():
52              msg = f"Directory ’{results_dir}’ does not exist! Run the current case in ‘calc‘
                    mode before executing ‘plot‘ mode."
53              raise FileNotFoundError(msg)
54
55          print(f"\n    Overwriting existing plots in results dir ’{results_dir}’")
56          plots.generate_report_plots(results_dir)
57          print(f"\n ‘PLOT‘ Execution of ‘{problem.problem_name}‘ Complete !!!")
58          return
59
60      if execution_mode == "calc":
61          if results_dir.exists():
62              print(f"XXX WARNING XXX The directory ’{results_dir}’ already exists!")
63              print("You will not be able to run in ’calc’ mode until this directory has a
                    different name.")
64              print("Either rename or delete the directory")
65              return
66
67      results_dir.mkdir(parents=True, exist_ok=True)
68      log_name = results_dir / f"{problem.problem_name}.log"
69
70      # executing calculation mode
71      print("\n!!! Executing in ‘CALC‘ !!!")
72      with open(log_name, "w") as log_file:
73          output_manager.print_problem_setup(problem, log_file)
74
75          mesh_dict = mesh_builder.discretize_space(problem)
76          quad_dict = quadrature.discretize_angle(problem)
```

```
77
78          msg = "Problem setup complete"
79          output_manager.log_time(start_time, msg, log_file)
80
81          msg = "\nBeginning to Solve the Fixed-Source Transport Problem using QD"
82          output_manager.log_msg(msg, log_file)
83
84          results_dict = solvers.qd_solver(
85              problem, mesh_dict, quad_dict, log_file
86          )
87
88          msg = "Completed Solving the Fixed-Source Transport Problem using QD"
89          output_manager.log_time(start_time, msg, log_file)
90
91          msg = "\nExporting Results"
92          output_manager.log_msg(msg, log_file)
93
94          output_manager.save_results(problem, results_dict, results_dir)
95
96      print(f"\n!!! ‘CALC‘ Execution of ‘{problem.problem_name}‘ Complete !!!")
97      return
98
99
100 if __name__ == "__main__":
101     main()
```

A.2 Spatial Mesh Discretization (`mesh_builder.py`)

```python
import numpy as np


def discretize_space(problem):
    region_edges   = np.array(problem.region_boundaries)
    region_widths  = region_edges[1:] - region_edges[:-1]
    delta_xs = np.array(problem.delta_xs)

    n_cells_per_region = np.round(region_widths / delta_xs).astype(int)
    n_cells_total = np.sum(n_cells_per_region)
    n_regions = len(region_widths)

    cell_edges = [region_edges[0]]
    for region_idx in range(n_regions):
        sub_edges = np.linspace(
            region_edges[region_idx],
            region_edges[region_idx + 1],
            n_cells_per_region[region_idx] + 1,
        )
        cell_edges.extend(sub_edges[1:])

    cell_edges   = np.array(cell_edges)
    cell_widths  = cell_edges[1:] - cell_edges[:-1]
    cell_centers = 0.5 * (cell_edges[1:] + cell_edges[:-1])

    sigma_t = np.zeros(n_cells_total)
    sigma_s = np.zeros(n_cells_total)
    sigma_f = np.zeros(n_cells_total)
    q       = np.zeros(n_cells_total)

    start_idx = 0
    for reg_idx in range(n_regions):
        end_idx = start_idx + n_cells_per_region[reg_idx]
        sigma_t[start_idx:end_idx] = problem.sigma_ts[reg_idx]
        sigma_s[start_idx:end_idx] = problem.sigma_ss[reg_idx]
        sigma_f[start_idx:end_idx] = problem.sigma_fs[reg_idx]
        q[start_idx:end_idx]       = problem.qs[reg_idx]
        start_idx = end_idx

```

```
    mesh_dict = {
        "n_cells": n_cells_total,
        "cell_edges": cell_edges,
        "cell_widths": cell_widths,
        "cell_centers": cell_centers,
        "sigma_t": sigma_t,
        "sigma_s": sigma_s,
        "sigma_f": sigma_f,
        "q": q,
    }

    return mesh_dict
```

A.3 Angular Quadrature Discretization (quadrature.py)

```python
import numpy as np


def discretize_angle(problem):
    order = problem.double_legendre_order

    xi, w_base = np.polynomial.legendre.leggauss(order)

    # negative half-range
    mu_neg = 0.5 * (xi - 1.0)
    w_neg  = 0.5 * w_base

    # positive half-range
    mu_pos = 0.5 * (xi + 1.0)
    w_pos  = 0.5 * w_base

    # combined
    mu = np.concatenate((mu_neg, mu_pos))
    w  = np.concatenate((w_neg, w_pos))

    quad_dict = {
        "n_angles": len(mu),
        "mu": mu,
        "w": w,
    }

    return quad_dict
```

A.4 Quasi-Diffusion and Transport Solvers (`solvers.py`)

```python
import numpy as np
from scipy.linalg import solve_banded


def qd_solver(problem, mesh_dict, quad_dict, log_file):
    epsilon = problem.convergence_criteria
    n_cells = mesh_dict["n_cells"]

    mu = quad_dict["mu"]
    w  = quad_dict["w"]

    pos_mask = mu > 0
    neg_mask = mu < 0

    # convergence history
    converged = False

    l_inf_history = []
    rho_history = [0.5, 0.5]

    # initial guesses (for s=0)
    phi_prev = np.ones(n_cells)
    phi_lld = np.ones((n_cells, 2))
    phi_error_prev = None

    E_avg = np.full(n_cells, 1.0 / 3.0)
    E_edge = np.full(n_cells + 1, 1.0 / 3.0)
    C_L = -1/2
    C_R = 1/2

    s = 0
    while not converged:
        s += 1

        if s > 1:
            # high-order LLD mesh sweep
            psi_avg, psi_edge = angular_mesh_sweep(problem, mesh_dict, quad_dict, phi_lld)

            # QD factors
```

```
        E_avg  = np.sum((mu[:, None] ** 2) * psi_avg * w[:, None], axis=0) / np.sum(
            psi_avg * w[:, None], axis=0)
        E_edge = np.sum((mu[:, None] ** 2) * psi_edge * w[:, None], axis=0) / np.sum(
            psi_edge * w[:, None], axis=0)

        # boundary factors
        C_L = np.sum(mu[neg_mask] * psi_edge[neg_mask, 0] * w[neg_mask]) / np.sum(
            psi_edge[neg_mask, 0] * w[neg_mask])
        C_R = np.sum(mu[pos_mask] * psi_edge[pos_mask, -1] * w[pos_mask]) / np.sum(
            psi_edge[pos_mask, -1] * w[pos_mask])

    # solve LOQD problem
    phi_avg, phi_edge, J_edge = fv_loqd_solver(
        problem, mesh_dict, quad_dict,
        E_avg, E_edge, C_L, C_R,
    )

    # apply discrete prolongation operator R_h for cell-edge, LLD-space phi
    phi_lld = np.zeros((n_cells, 2))
    phi_lld[:, 0] = 0.5 * phi_edge[:-1] + phi_avg - 0.5 * phi_edge[1:] # left
    phi_lld[:, 1] = -0.5 * phi_edge[:-1] + phi_avg + 0.5 * phi_edge[1:] # right

    # convergence checks
    l_inf = np.max(np.abs(phi_avg - phi_prev))
    l_inf_history.append(l_inf)

    if s > 2 and phi_error_prev > 0.0:
        rho = l_inf / phi_error_prev
        rho_history.append(rho)
    else:
        rho = rho_history[s - 1]

    msg = f"Iter {s:3d} | L_inf = {l_inf:.6e} | rho = {rho:.6f}\n"
    log_file.write(msg)

    if s >= 2:
        tolerence_criteria = epsilon * (1.0 / rho - 1.0)

        if l_inf <= tolerence_criteria:
            converged = True

```

```
        # update for next iteration
        phi_error_prev = l_inf
        phi_prev = phi_avg.copy()

    # sweep with converged values
    psi_avg, psi_edge = angular_mesh_sweep(problem, mesh_dict, quad_dict, phi_lld)
    E_avg, E_edge, C_L, C_R = compute_qd_factors(quad_dict, psi_avg, psi_edge)

    # transport metrics
    M_E = np.abs(1.0 / 3.0 - E_avg)
    M_E_prime = np.abs((E_edge[1:] - E_edge[:-1]) / mesh_dict["cell_widths"])

    results_dict = {
        # discretization variables
        "cell_centers": mesh_dict["cell_centers"],
        "cell_edges": mesh_dict["cell_edges"],
        "cell_widths": mesh_dict["cell_widths"],
        "mu": quad_dict["mu"],
        # solution grid functions
        "psi_avg": psi_avg,
        "phi_avg": phi_avg,
        "phi_edge": phi_edge,
        "J_edge": J_edge,
        # QD closure and boundary factors
        "E_avg": E_avg,
        "E_edge": E_edge,
        "C_L": C_L,
        "C_R": C_R,
        # transport metrics
        "M_E": M_E,
        "M_E_prime": M_E_prime,
        # convergence
        "l_inf_history": np.array(l_inf_history),
        "rho_history": np.array(rho_history),
        "n_iterations": s,
    }

    return results_dict


def angular_mesh_sweep(problem, mesh_dict, quad_dict, phi_lld_prev):
```

```
    n_cells = mesh_dict["n_cells"]
    cell_widths = mesh_dict["cell_widths"]
    sigma_t = mesh_dict["sigma_t"]
    sigma_s = mesh_dict["sigma_s"]
    q = mesh_dict["q"]

    mu_arr = quad_dict["mu"]
    w_arr  = quad_dict["w"]

    phi_L = phi_lld_prev[:, 0]
    phi_R = phi_lld_prev[:, 1]

    n_angles = len(mu_arr)
    psi_avg  = np.zeros((n_angles, n_cells))
    psi_edge = np.zeros((n_angles, n_cells + 1))

    for m_idx, mu in enumerate(mu_arr):
        w = w_arr[m_idx]

        # forward sweep using positive half-range equations
        if mu > 0:
            if problem.left_bc == "vacuum":
                psi_incoming = 0.0
            if problem.left_bc == "incident":
                psi_incoming = problem.left_incident_flux

            psi_edge[m_idx, 0] = psi_incoming

            for cell_idx in range(n_cells):
                dx = cell_widths[cell_idx]

                # forcing matrix
                a11 = 0.5 * (mu + sigma_t[cell_idx] * dx)
                a12 = 0.5 * (mu)
                a21 = 0.5 * (-mu)
                a22 = 0.5 * (mu + sigma_t[cell_idx] * dx)

                # source vector
                f1 = dx/4 * (sigma_s[cell_idx] * phi_L[cell_idx] + q[cell_idx]) + mu *
                    psi_incoming
                f2 = dx/4 * (sigma_s[cell_idx] * phi_R[cell_idx] + q[cell_idx])
```

```
158
159                 # manually computer inverse
160                 det = a11 * a22 - a12 * a21
161                 psi_L = (f1 * a22 - f2 * a12) / det
162                 psi_R = (f2 * a11 - f1 * a21) / det
163
164                 psi_avg[m_idx, cell_idx] = 0.5 * (psi_L + psi_R)
165                 psi_edge[m_idx, cell_idx + 1] = psi_R
166
167                 # incoming flux for next cell
168                 psi_incoming = psi_R
169             continue
170
171         # backward sweep using negative half-range equations
172         if mu < 0:
173             if problem.right_bc == "vacuum":
174                 psi_incoming = 0.0
175             if problem.right_bc == "incident":
176                 psi_incoming = problem.right_incident_flux
177
178             psi_edge[m_idx, n_cells] = psi_incoming
179
180             for cell_idx in range(n_cells - 1, -1, -1):
181                 dx = cell_widths[cell_idx]
182
183                 # forcing matrix
184                 a11 = 0.5 * (-mu + sigma_t[cell_idx] * dx)
185                 a12 = 0.5 * (mu)
186                 a21 = 0.5 * (-mu)
187                 a22 = 0.5 * (-mu + sigma_t[cell_idx] * dx)
188
189                 # source vector
190                 f1 = dx/4 * (sigma_s[cell_idx] * phi_L[cell_idx] + q[cell_idx])
191                 f2 = dx/4 * (sigma_s[cell_idx] * phi_R[cell_idx] + q[cell_idx]) - mu *
                        psi_incoming
192
193                 # manually computer inverse
194                 det = a11 * a22 - a12 * a21
195                 psi_L = (f1 * a22 - f2 * a12) / det
196                 psi_R = (f2 * a11 - f1 * a21) / det
197
```

```
                psi_avg[m_idx, cell_idx] = 0.5 * (psi_L + psi_R)
                psi_edge[m_idx, cell_idx] = psi_L

                # incoming flux for next cell
                psi_incoming = psi_L
            continue

    return psi_avg, psi_edge


def fv_loqd_solver(problem, mesh_dict, quad_dict, E_avg, E_edge, C_L, C_R):
    n_cells = mesh_dict["n_cells"]
    dx      = mesh_dict["cell_widths"]
    sigma_t = mesh_dict["sigma_t"]
    sigma_s = mesh_dict["sigma_s"]
    q       = mesh_dict["q"]
    sigma_a = sigma_t - sigma_s

    mu = quad_dict["mu"]
    w  = quad_dict["w"]

    # compute incoming partial fluxes and currents
    pos_mask = mu > 0
    neg_mask = mu < 0

    if problem.left_bc == "vacuum":
        phi_in_pos = 0.0
        J_in_pos   = 0.0
    if problem.left_bc == "incident":
        psi_L_in   = problem.left_incident_flux
        phi_in_pos = np.sum(psi_L_in * w[pos_mask])
        J_in_pos   = np.sum(mu[pos_mask] * psi_L_in * w[pos_mask])

    if problem.right_bc == "vacuum":
        phi_in_neg = 0.0
        J_in_neg   = 0.0
    elif problem.right_bc == "incident":
        psi_R_in   = problem.right_incident_flux
        phi_in_neg = np.sum(psi_R_in * w[neg_mask])
        J_in_neg   = np.sum(mu[neg_mask] * psi_R_in * w[neg_mask])

```

```
    # assemble matrix A
    a_banded = np.zeros((3, n_cells))
    f = q * dx

    tau = sigma_t * dx
    denom_adjacent = tau[:-1] + tau[1:]

    # off-diagonals
    a_banded[0, 1:]  = -2.0 * E_avg[1:] / denom_adjacent   # super-diag
    a_banded[2, :-1] = -2.0 * E_avg[:-1] / denom_adjacent  # sub-diag

    # main diagonal
    a_banded[1, 1:-1] = (
        2.0 * E_avg[1:-1] / denom_adjacent[:-1]
        + sigma_a[1:-1] * dx[1:-1]
        + 2.0 * E_avg[1:-1] / denom_adjacent[1:]
    )

    # left BC
    denom_L = E_edge[0] - 0.5 * tau[0] * C_L
    a_banded[1, 0] = (2.0 * E_avg[0] / denom_adjacent[0]) + (C_L * E_avg[0] / denom_L) +
        sigma_a[0] * dx[0]
    f[0] = q[0] * dx[0] + (E_edge[0] * (J_in_pos - C_L * phi_in_pos)) / denom_L

    # right BC
    denom_R = E_edge[-1] + 0.5 * tau[-1] * C_R
    a_banded[1, -1] = (2.0 * E_avg[-1] / denom_adjacent[-1]) + (C_R * E_avg[-1] / denom_R) +
         sigma_a[-1] * dx[-1]
    f[-1] = q[-1] * dx[-1] - (E_edge[-1] * (J_in_neg - C_R * phi_in_neg)) / denom_R

    phi_avg = solve_banded((1, 1), a_banded, f)

    # edge quantities
    phi_edge = np.zeros(n_cells + 1)
    J_edge = np.zeros(n_cells + 1)

    # interior cells
    phi_edge[1:-1] = (
        tau[1:] * E_avg[:-1] * phi_avg[:-1] + tau[:-1] * E_avg[1:] * phi_avg[1:]
    ) / (E_edge[1:-1] * denom_adjacent)

```

```
    J_edge[1:-1] = (2.0 / denom_adjacent) * (E_avg[:-1] * phi_avg[:-1] - E_avg[1:] * phi_avg
        [1:])

    # left bc
    phi_edge[0] = (E_avg[0] * phi_avg[0] + 0.5 * tau[0] * (J_in_pos - C_L * phi_in_pos)) /
        denom_L
    J_edge[0]   = (C_L * E_avg[0] * phi_avg[0] + E_edge[0] * (J_in_pos - C_L * phi_in_pos))
        / denom_L

    # right bc
    phi_edge[-1] = (E_avg[-1] * phi_avg[-1] - 0.5 * tau[-1] * (J_in_neg - C_R * phi_in_neg))
         / denom_R
    J_edge[-1]   = (C_R * E_avg[-1] * phi_avg[-1] + E_edge[-1] * (J_in_neg - C_R *
        phi_in_neg)) / denom_R

    return phi_avg, phi_edge, J_edge


def compute_qd_factors(quad_dict, psi_avg, psi_edge):
    mu = quad_dict["mu"]
    w  = quad_dict["w"]

    pos_mask = mu > 0
    neg_mask = mu < 0

    w_col  = w[:, None]
    mu_col = mu[:, None]

    # QD factors
    E_avg  = np.sum((mu_col ** 2) * psi_avg * w_col, axis=0) / np.sum(psi_avg * w_col, axis
        =0)
    E_edge = np.sum((mu_col ** 2) * psi_edge * w_col, axis=0) / np.sum(psi_edge * w_col,
        axis=0)

    # boundary factors
    C_L = np.sum(mu[neg_mask] * psi_edge[neg_mask, 0] * w[neg_mask]) / np.sum(psi_edge[
        neg_mask, 0] * w[neg_mask])
    C_R = np.sum(mu[pos_mask] * psi_edge[pos_mask, -1] * w[pos_mask]) / np.sum(psi_edge[
        pos_mask, -1] * w[pos_mask])

    return E_avg, E_edge, C_L, C_R
```

A.5 Output and Logging Manager (output_manager.py)

```python
from pathlib import Path
import numpy as np
import time


def print_problem_setup(problem, log_file):
    lines = [
        "=" * 50,
        f"INITIALIZING SOLVER FOR '{problem.problem_name}'",
        "=" * 50,
        f"  Region Boundaries:  {problem.region_boundaries}",
        f"  Total Cross Sections:  {problem.sigma_ts}",
        f"  Scattering Cross Sections:  {problem.sigma_ss}",
        f"  Fission Cross Sections:  {problem.sigma_fs}",
        f"  External Sources:  {problem.qs}",
        f"  Left Boundary Condition:  {problem.left_bc}",
    ]

    if hasattr(problem, "left_incident_flux"):
        msg = f"    Left Incident Flux:      {problem.left_incident_flux}"
        lines.append(msg)

    lines.extend([
        f"  Right Boundary Condition:  {problem.right_bc}",
        f"  Delta xs: {problem.delta_xs}",
        f"  Double Legendre Order:  {problem.double_legendre_order}",
        f"  Convergence Criteria:      {problem.convergence_criteria}",
    ])

    [log_msg(line, log_file) for line in lines]
    return


def log_msg(msg, file_obj):
    print(msg)
    file_obj.write(msg + "\n")
    return

def log_time(start_time, msg, file_obj):
```

```
    elapsed_time = time.time() - start_time
    full_msg = f"{msg} (Elapsed Time: {elapsed_time:.4f} seconds)"
    log_msg(full_msg, file_obj)
    return


def save_results(problem, results_dict, results_dir):
    problem_name = problem.problem_name

    # binary for --plot mode!!
    np.savez(results_dir / f"{problem_name}_results.npz", **results_dict)

    # saving formatted values
    save_cell_average_quantities(results_dir, problem_name, results_dict)
    save_cell_edge_quantities(results_dir, problem_name, results_dict)
    save_angular_flux(results_dir, problem_name, results_dict)
    save_convergence_summary(results_dir, problem_name, results_dict)


def save_cell_average_quantities(results_dir, problem_name, results_dict):
    x_i       = results_dict["cell_centers"]
    phi_avg   = results_dict["phi_avg"]
    E_avg     = results_dict["E_avg"]
    M_E       = results_dict["M_E"]
    M_E_prime = results_dict["M_E_prime"]

    out_file = results_dir / f"{problem_name}_cell_averages.txt"
    with open(out_file, "w") as f:
        f.write(f"CELL-AVERAGE QUANTITIES FOR {problem_name}\n")
        f.write(
            f"{'Cell idx':<10s}{'x_i (cm)':<12s}"
            f"{'phi_i':<16s}{'E_i':<16s}"
            f"{'M^E_i':<16s}{'M^E_prime_i':<16s}\n"
        )
        for i in range(len(x_i)):
            f.write(
                f"{i:<10d}{x_i[i]:<12.5f}"
                f"{phi_avg[i]:<16.6e}{E_avg[i]:<16.6e}"
                f"{M_E[i]:<16.6e}{M_E_prime[i]:<16.6e}\n"
            )

```

```
81
82  def save_cell_edge_quantities(results_dir, problem_name, results_dict):
83      x_edge   = results_dict["cell_edges"]
84      phi_edge = results_dict["phi_edge"]
85      J_edge   = results_dict["J_edge"]
86      E_edge   = results_dict["E_edge"]
87
88      out_file = results_dir / f"{problem_name}_cell_edges.txt"
89      with open(out_file, "w") as f:
90          f.write(f"CELL-EDGE QUANTITIES FOR {problem_name}\n")
91          f.write(
92              f"{'Edge idx':<10s}{'x_i+1/2 (cm)':<14s}"
93              f"{'phi_i+1/2':<16s}{'J_i+1/2':<16s}{'E_i+1/2':<16s}\n"
94          )
95          for i in range(len(x_edge)):
96              f.write(
97                  f"{i:<10d}{x_edge[i]:<14.5f}"
98                  f"{phi_edge[i]:<16.6e}{J_edge[i]:<16.6e}{E_edge[i]:<16.6e}\n"
99              )
100
101
102 def save_angular_flux(results_dir, problem_name, results_dict):
103     x_i      = results_dict["cell_centers"]
104     mu       = results_dict["mu"]
105     psi_avg = results_dict["psi_avg"]
106
107     out_file = results_dir / f"{problem_name}_cell_average_angular_flux.txt"
108     with open(out_file, "w") as f:
109         f.write(f"CELL-AVERAGE ANGULAR FLUX psi_{{i,m}} FOR {problem_name}\n")
110         f.write(f"{'Angle m':<10s}{'mu_m':<12s}")
111         for x in x_i:
112             f.write(f"{'x=' + f'{x:.2f}':<14s}")
113         f.write("\n" + "-" * (22 + 14 * len(x_i)) + "\n")
114
115         for m in range(len(mu)):
116             f.write(f"{m:<10d}{mu[m]:<+12.6f}")
117             for i in range(len(x_i)):
118                 f.write(f"{psi_avg[m, i]:<14.6e}")
119             f.write("\n")
120
121
```

```python
def save_convergence_summary(results_dir, problem_name, results_dict):
    l_inf_hist = results_dict["l_inf_history"]
    rho_hist   = results_dict["rho_history"]
    n_iter     = results_dict["n_iterations"]
    C_L        = results_dict["C_L"]
    C_R        = results_dict["C_R"]

    out_file = results_dir / f"{problem_name}_convergence_and_summary.txt"
    with open(out_file, "w") as f:
        f.write(f"CONVERGENCE & BOUNDARY SUMMARY FOR {problem_name}\n")
        f.write("=" * 55 + "\n")
        f.write(f"Total Iterations (s):      {n_iter}\n")
        f.write(f"Left Boundary Factor C_L:  {C_L:.8f}\n")
        f.write(f"Right Boundary Factor C_R: {C_R:.8f}\n")
        f.write("=" * 55 + "\n\n")
        f.write(f"{'Iter s':<10s}{'||phi^(s) - phi^(s-1)||_inf':<28s}{'Est. Spectral Radius rho^(s)':<28s}\n")
        for idx in range(n_iter):
            f.write(f"{idx + 1:<10d}{l_inf_hist[idx]:<28.6e}{rho_hist[idx]:<28.6f}\n")
```

A.6 Report Plotting Module (plots.py)

```python
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path


def generate_report_plots(results_dir):
    results_dir = Path(results_dir)
    plots_dir = results_dir / "plots"
    plots_dir.mkdir(parents=True, exist_ok=True)

    npz_path = next(results_dir.glob("*.npz"))
    data = np.load(npz_path)

    plot_angular_flux_2d(data, plots_dir)
    plot_cell_avg_scalar_flux(data, plots_dir)
    plot_cell_edge_scalar_flux(data, plots_dir)
    plot_cell_edge_current(data, plots_dir)
    plot_cell_avg_qd_factor(data, plots_dir)
    plot_cell_edge_qd_factor(data, plots_dir)
    plot_departure_from_diffusion_metric(data, plots_dir)
    plot_qd_gradient_metric(data, plots_dir)
    plot_convergence_history(data, plots_dir)
    plot_spectral_radius(data, plots_dir)


def plot_angular_flux_2d(data, plots_dir):
    x_i = data["cell_centers"]
    mu = data["mu"]
    psi_avg = data["psi_avg"]

    fig, ax = plt.subplots(figsize=(7.5, 4.8))
    mesh = ax.pcolormesh(x_i, mu, psi_avg, shading="nearest", cmap="viridis")
    fig.colorbar(mesh, ax=ax, label=r"$\psi_{i,m}$")

    ax.set_xlabel(r"$x_i$ (cm)")
    ax.set_ylabel(r"$\mu_m$")

    fig.tight_layout()
    fig.savefig(plots_dir / "01_angular_flux.png", dpi=600)
```

```
    plt.close(fig)


def plot_cell_avg_scalar_flux(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(data["cell_centers"], data["phi_avg"], linewidth=1.6)
    ax.set_xlabel(r"$x_i$ (cm)")
    ax.set_ylabel(r"$\phi_i$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "02_phi_avg.png", dpi=600)
    plt.close(fig)


def plot_cell_edge_scalar_flux(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(data["cell_edges"], data["phi_edge"], linewidth=1.6)
    ax.set_xlabel(r"$x_{i+1/2}$ (cm)")
    ax.set_ylabel(r"$\phi_{i+1/2}$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "03_phi_edge.png", dpi=600)
    plt.close(fig)


def plot_cell_edge_current(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(data["cell_edges"], data["J_edge"], linewidth=1.6)
    ax.set_xlabel(r"$x_{i+1/2}$ (cm)")
    ax.set_ylabel(r"$J_{i+1/2}$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "04_J_edge.png", dpi=600)
    plt.close(fig)


def plot_cell_avg_qd_factor(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.axhline(1.0 / 3.0, color="r", linestyle="--", linewidth=1.1, label=r"Diffusion Limit
        ($1/3$)")
    ax.plot(data["cell_centers"], data["E_avg"], linewidth=1.6, label=r"$E_i$")
```

```
    ax.set_xlabel(r"$x_i$ (cm)")
    ax.set_ylabel(r"$E_i$")
    ax.legend()
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "05_E_avg.png", dpi=600)
    plt.close(fig)


def plot_cell_edge_qd_factor(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.axhline(1.0 / 3.0, color="r", linestyle="--", linewidth=1.1, label=r"Diffusion Limit
        ($1/3$)")
    ax.plot(data["cell_edges"], data["E_edge"], linewidth=1.6, label=r"$E_{i+1/2}$")
    ax.set_xlabel(r"$x_{i+1/2}$ (cm)")
    ax.set_ylabel(r"$E_{i+1/2}$")
    ax.legend()
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "06_E_edge.png", dpi=600)
    plt.close(fig)


def plot_departure_from_diffusion_metric(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(data["cell_centers"], data["M_E"], linewidth=1.6)
    ax.set_xlabel(r"$x_i$ (cm)")
    ax.set_ylabel(r"$M_i^E = |1/3 - E_i|$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "07_metric_ME.png", dpi=600)
    plt.close(fig)


def plot_qd_gradient_metric(data, plots_dir):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(data["cell_centers"], data["M_E_prime"], linewidth=1.6)
    ax.set_xlabel(r"$x_i$ (cm)")
    ax.set_ylabel(r"$M_i^{E'} = |(E_{i+1/2} - E_{i-1/2}) / \Delta x_i|$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
```

```
    fig.savefig(plots_dir / "08_metric_ME_prime.png", dpi=600)
    plt.close(fig)


def plot_convergence_history(data, plots_dir):
    l_inf_hist = data["l_inf_history"]
    iters = np.arange(1, len(l_inf_hist) + 1)

    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.semilogy(iters, l_inf_hist, linewidth=1.6)
    ax.set_xlabel(r"Iteration Index $s$")
    ax.set_ylabel(r"$\|\phi^{(s)} - \phi^{(s-1)}\|_\infty$")
    ax.grid(True, which="both", linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "09_convergence_history.png", dpi=600)
    plt.close(fig)


def plot_spectral_radius(data, plots_dir):
    rho_hist = data["rho_history"][2:]
    iters = np.arange(3, len(rho_hist) + 3)

    fig, ax = plt.subplots(figsize=(7, 4.5))
    style = "o" if len(iters) == 1 else "-"
    ax.plot(iters, rho_hist, style, linewidth=1.6)
    ax.set_xlabel(r"Iteration Index $s$")
    ax.set_ylabel(r"$\rho^{(s)}$")
    ax.grid(True, linestyle="--", alpha=0.6)
    fig.tight_layout()
    fig.savefig(plots_dir / "10_spectral_radius.png", dpi=600)
    plt.close(fig)
```

B Input Files

B.1 Test A (input_files/test_a.py)

```
problem_name = "test_a"

region_boundaries = [0, 1, 11]
sigma_ts = [2, 100]
sigma_ss = [0, 100]
sigma_fs = [0,   0]
qs = [0, 0]

left_bc = "incident"
left_incident_flux = 1
right_bc = "vacuum"

delta_xs = [0.1, 1]

double_legendre_order = 4

convergence_criteria = 1e-10
```

B.2 Test B (input_files/test_b.py)

```
problem_name = "test_b"

region_boundaries = [0, 10, 30]
sigma_ts = [  1,  10]
sigma_ss = [0.5, 9.9]
sigma_fs = [  0,   0]
qs = [0.5, 0.01]

left_bc = "vacuum"
right_bc = "vacuum"

delta_xs = [1, 0.5]

double_legendre_order = 4

convergence_criteria = 1e-10
```

B.3 Test C (input_files/test_c.py)

```
problem_name = "test_c"

region_boundaries = [0, 10, 30]
sigma_ts = [  1,  10]
sigma_ss = [0.5, 9.9]
sigma_fs = [  0,   0]
qs = [0.5, 0]

left_bc = "vacuum"
right_bc = "vacuum"

delta_xs = [1, 0.5]

double_legendre_order = 4

convergence_criteria = 1e-10
```

B.4 Test D(a) (input_files/test_d1.py)

```
problem_name = "test_d1"

# calcs
nu = 1e-2

sigma_t = 1 / nu
sigma_a = nu
sigma_s = sigma_t - sigma_a



# variables to import
region_boundaries = [0, 1]
sigma_ts = [sigma_t]
sigma_ss = [sigma_s]
sigma_fs = [0]
qs = [nu]

left_bc = "vacuum"
right_bc = "vacuum"

```

```
delta_xs = [0.1]

double_legendre_order = 4

convergence_criteria = 1e-10
```

B.5 Test D(b) (input_files/test_d2.py)

```
problem_name = "test_d2"

# calcs
nu = 1e-4

sigma_t = 1 / nu
sigma_a = nu
sigma_s = sigma_t - sigma_a



# variables to import
region_boundaries = [0, 1]
sigma_ts = [sigma_t]
sigma_ss = [sigma_s]
sigma_fs = [0]
qs = [nu]

left_bc = "vacuum"
right_bc = "vacuum"

delta_xs = [0.1]

double_legendre_order = 4

convergence_criteria = 1e-10
```